

International University
School of Computer Science and Engineering



WEB APPLICATION DEVELOPMENT T
PROJECT REPORT
IU FORUM

Lab 02 Semester: 2, Year: 2025 – 2026

Lab Instructor: Dr. N.T.Nghia

Student information

Đồng Đức Dũng - ITCSIU24023

Lê Nguyễn Hải Đăng - ITCSIU24017

Table of Contents

I. INTRODUCTION	6
1. Project Overview	6
2. Problem Statement	6
3. Objectives	7
4. Scope of the Project	7
5. Technology Stack	8
6. Deployment URL & Test Credentials	8
II. Requirements Analysis	9
1. Problem Statement	9
2. User Stories	10
3. Functional Requirements	13
3.1.FR-01 — Authentication and Account Management	13
3.2.FR-02 — Thread Management	13
3.3.FR-03 — Message (Reply) Management	13
3.4.FR-04 — Search and Filtering	14
3.5.FR-05 — User Profile Management	14
3.6.FR-06 — Moderation	15
3.7.FR-07 — Administration	15
3.8.FR-08 — Automated Retention Policy	15
4. Non-Functional Requirements	15
4.1.Performance	15
4.2.Security	16
4.3.Usability	16
4.4.Scalability	17
4.5.Reliability	17
III. System Architecture	17
1. Architecture Overview	17
2. Technology Stack Justification	18
2.1.Frontend — Thymeleaf + Vanilla JavaScript vs. React / Vue.js	18
2.2.Backend — Spring Boot 4.0.6 vs. Raw Java Servlets	19

2.3.Database — MySQL 8.0 vs. MongoDB.....	20
2.4.ORM — Spring Data JPA (Hibernate) vs. Raw JDBC	20
2.5.Authentication — Spring Security vs. Custom Session Management	21
2.6.Deployment — Railway vs. Heroku / AWS EC2.....	21
3. Design Patterns	22
3.1.Model-View-Controller (MVC).....	22
3.2.Repository Pattern	22
3.3.Singleton Pattern	23
3.4.Soft Delete Pattern	23
3.5.Template Method Pattern (JPA Lifecycle Hooks).....	24
3.6.Factory Pattern (Spring IoC Container).....	24
4. API Design	24
4.1 RESTful principles followed.....	25
4.2 URL structure	25
4.3 HTTP methods usage.....	25
4.4 Status codes strategy	25
4.5 Request/response examples for key endpoints	26
IV. Database Design	27
1. Entity-Relationship Diagram.....	27
2. Table Schemas	29
2.1. TABLE: users.....	29
2.2. TABLE: categories	29
2.3. TABLE: threads	30
2.4. TABLE: messages	30
2.5. TABLE: tags.....	31
2.6. TABLE: thread_tags (Junction).....	31
2.7. TABLE: thread_likes (Junction)	31
2.8. TABLE: verification_tokens.....	32
3. Relationships Explanation	32
4. Normalization Analysis	33
5. Indexing Strategy.....	34

V. Implementation Details	36
1. Key Algorithms & Business Logic	36
Algorithm 1 — Multi-criteria Advanced Filter.....	36
Algorithm 2 — Two-Phase Content Retention Policy.....	37
Algorithm 3 — Tag Auto-Create on Thread Creation	37
Algorithm 4 — Email Verification Token Flow	38
Algorithm 5 — Multi-scope Search with Dynamic Branching	39
2. Security Implementation	39
3. Error Handling Strategy	40
3.1.Error Handling Strategy.....	41
3.2.Controller-Level Business Rule Error Mapping.....	41
3.3.Bean Validation Examples.....	41
3.4.File Upload Error Handling Flow.....	42
3.5.Database Error Handling	42
3.6.Logging Strategy.....	42
4. Advanced Features Implementation	42
Feature 1 — File Upload System.....	42
Feature 2 — Moderator Dashboard with Chart.js Analytics	43
Feature 3 — Automated Content Retention (Scheduled Jobs)	44
Feature 4 — Multi-criteria Search and Filter System	44
Feature 5 — Remember Me / Persistent Login	44
5. AI Assistance Report.....	45
5.1. AI Contribution by Area	45
5.2. Overall Estimated Contribution	46
5.3. Major Components Where AI Was Used	46
5.4.Challenges with AI-Generated Code.....	50
Overall Impact	51
VI. Testing & Quality Assurance.....	51
1. Testing Strategy	51
2. Bug Tracking	54
3. Known Limitations.....	56

VII. Deployment	58
1. Deployment Process	58
2. CI/CD Pipeline	59
3. Monitoring & Maintenance	60
4. Monitoring & Maintenance	60
VIII. User Guide	61
1. Getting Started	61
1.1. Test Account Credentials:	61
1.2. Accessing the Application	61
1.3. Registering a New Account	61
1.4. Logging In	64
2. Feature Walkthroughs	65
2.1. Browsing and Searching Threads (All Users)	65
2.2. Creating a Thread (Logged-in Users)	66
2.3. Replying to a Thread and Uploading Files (Logged-in Users)	68
2.4. Moderator Dashboard (ROLE_MODERATOR and ROLE_ADMIN only)	69
2.5. Admin User Management (ROLE_ADMIN only)	70
IX. Team Collaboration	72
1. Team Organization	72
2. Individual Contributions	73
3. Challenges in Teamwork	73
X. Conclusion & Reflection	74
1. Project Summary	74
2. Lessons Learned	75
3. Individual Reflections	75
4. Future Enhancements	76
Appendices	77
Appendix A: Meeting Logs	77
Appendix B: API Documentation	77
Appendix C: Git Statistics	79
Appendix D: User Feedback	79

I. INTRODUCTION

1. Project Overview

IU Forum is a web-based discussion platform designed for students and lecturers in an academic environment. The system enables users to create discussion threads, share knowledge, ask questions, and interact with one another across clearly defined topic categories.

This project was developed as the final deliverable for the **Web Application Development** course, applying all concepts covered from Lab 01 through Lab 08 to build a complete full-stack system — from database design and backend logic to frontend interface and production deployment.

2. Problem Statement

University students currently face significant challenges when it comes to exchanging subject-specific knowledge in an organized manner. General-purpose

messaging platforms such as Messenger or Zalo lack the ability to categorize topics, search historical content, or enforce role-based access control. As a result, knowledge becomes fragmented, questions are repeatedly asked without reference to previous answers, and there is no moderation mechanism to ensure content quality.

IU Forum addresses these problems by providing a structured discussion platform where content is organized by category, access is managed through a role-based permission system, and full-text search allows users to quickly locate relevant information.

3. Objectives

The system serves three primary user groups within the university environment:

- **Students (User role):** The primary user group, who can create threads, post comments, search for content, and bookmark topics of interest. This group generates the majority of content on the platform.
- **Moderators:** Designated students or teaching assistants who are granted elevated permissions to manage content within assigned categories — including locking threads, removing violating comments, and pinning important announcements.
- **Administrators (Admin role):** Hold full system privileges, including user account management, category administration, system-wide content oversight, and configuration control.

4.Scope of the Project

The system delivers the following core features:

- **User Management:** Account registration with email verification, secure login/logout with Spring Security, personal profile management, and role-based access control across three distinct roles (Admin, Moderator, User)
- **Thread Management:** Create, edit, and soft-delete discussion threads; support for tag assignment and sticky pinning for important announcements
- **Comment System:** Post, edit, and soft-delete replies within threads; support for file attachments
- **Advanced Search & Filtering:** Full-text keyword search on thread titles, filtering by category, status, and tag; pagination with configurable page sizes (10, 20, 50 items)
- **In-App Notification System:** Real-time notifications for four event types — REPLY, MENTION, SYSTEM, and MOD_ACTION — with unread badge counts

- **Personal Bookmarks:** Save and organize favorite threads into named folders for quick access
- **Admin Dashboard:** Dedicated interface for administrators to manage users, categories, and platform-wide content

5. Technology Stack

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript	Build a visual interface, display forum content, and handle user interactions on the client side
Backend	Java, Spring Boot, Spring Web	Handle business logic, process and route HTTP requests, and act as a bridge between the user interface and the database.
Database	MySQL	Persistently store and manage the structured data of the system
Database access	Spring Data JPA (Hibernate)	Serve as an ORM (Object-Relational Mapping) bridge between Java code and MySQL.
Authentication	Spring Security	Control access rights, authenticate user identity (Login/Logout), encrypt passwords, and manage authorization
Deployment	Railway (Cloud Host) GitHub (CI/CD)	Package and deploy the application to a public Internet server. Automate the CI/CD pipeline to deploy code changes from GitHub to the production environment.

6. Deployment URL & Test Credentials

Live URL: <https://forum-production-028c.up.railway.app/>

Role	Username	Password
Admin	admin	123456
Moderator	mod	123456
User	student_dung/student_dang	123456

II. Requirements Analysis

1. Problem Statement

The Problem: General platforms (social media, chat apps) lack hierarchical organization, persistent threading, and content governance. This leads to fragmented conversations, difficult knowledge retrieval, and poor content quality in academic and interest-based communities.

The Solution: The Forum Web Application is a structured, category-based discussion platform tailored for university students and online learning communities.

Target users and their pain points are identified as follows:

- **Guest / Visitor** — Individuals who encounter the forum via search engines or shared links. Pain point: they cannot contribute or receive notifications without registering, and the registration process must be frictionless enough not to deter participation.
- **Member** — Registered students or community participants who wish to ask questions, share knowledge, and engage in discussions. Pain points include difficulty locating relevant threads in a growing content base, lack of awareness when monitored threads receive new replies, and frustration when posted content cannot be corrected after submission.
- **Moderator** — Trusted community members assigned to uphold content standards. Pain points include the time-consuming manual review of flagged content, the risk of permanently deleting messages that may be referenced later, and the absence of at-a-glance metrics showing forum activity trends.
- **Administrator** — The technical owner responsible for platform governance. Pain points include the need to manage role assignments, control user access, and configure system-wide behaviour without modifying application code or redeploying the service.

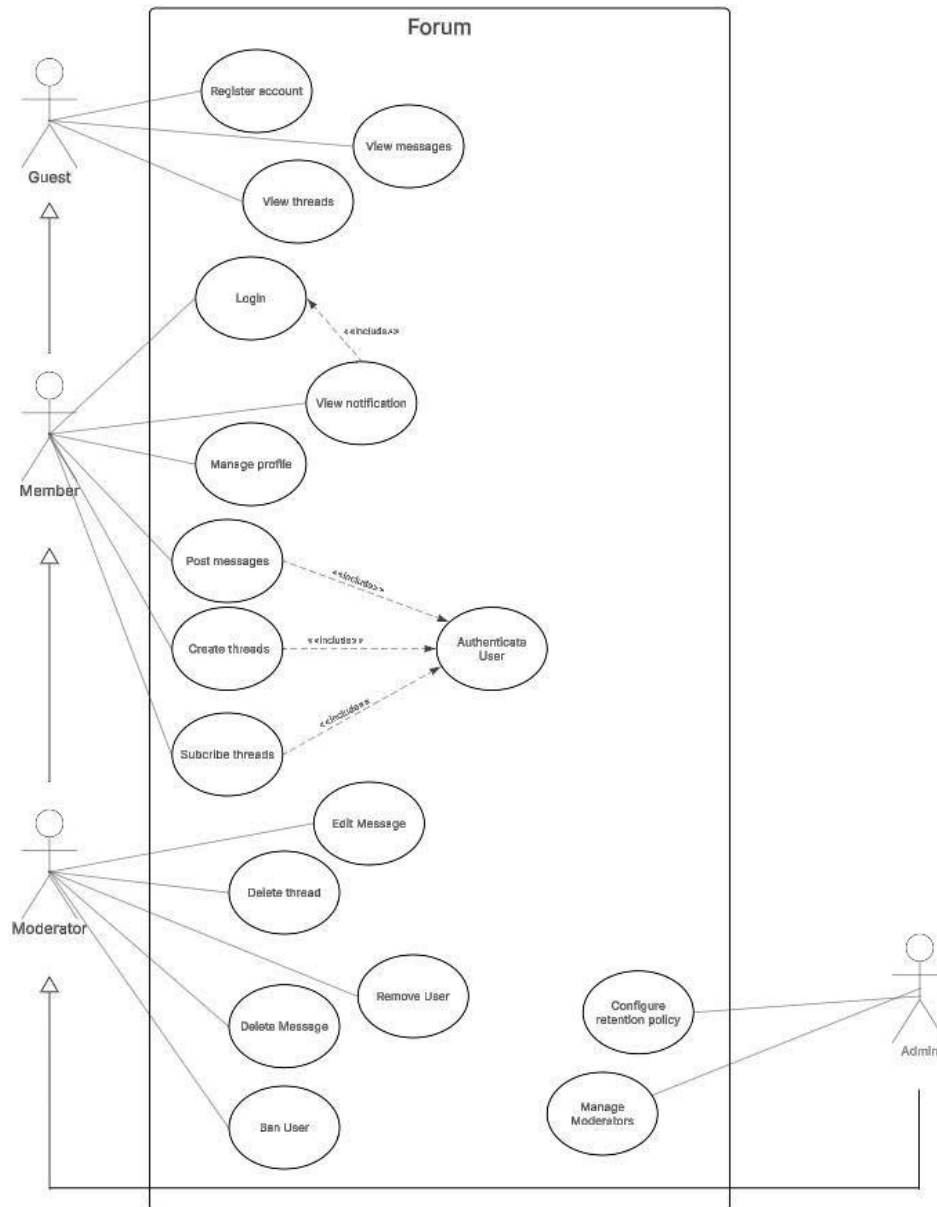


Figure 2.1: Use-Case Diagram

2. User Stories

The following user stories capture the functional needs of all four actor roles. Each story is prioritised using the MoSCoW framework: Must Have (core system functionality), Should Have (important but not blocking), and Nice to Have (desirable enhancements).

ID	Role	User Story	Priority
----	------	------------	----------

US-01	Guest	As a Guest , I want to browse all public discussion threads on the homepage, so that I can read community content before deciding to register.	Must Have
US-02	Guest	As a Guest , I want to view the full content of a thread and its replies, so that I can get answers without needing an account.	Must Have
US-03	Guest	As a Guest , I want to register a new account with a username, email, and password, so that I can become a Member of the forum.	Must Have
US-04	Guest	As a Guest , I want to receive an email verification link after registering, so that my account is activated securely before I can log in.	Must Have
US-05	Member	As a Member , I want to log in with my username and password, so that I can access authenticated features of the forum.	Must Have
US-06	Member	As a Member , I want to choose "Remember Me" when logging in, so that I remain logged in for up to 30 days without re-entering my credentials.	Should Have
US-07	Member	As a Member , I want to create a new discussion thread by providing a title, category, content, and optional tags, so that I can start a conversation on a topic.	Must Have
US-08	Member	As a Member , I want to attach a file (image, PDF, or document) when posting a message, so that I can share supporting material with my reply.	Should Have
US-09	Member	As a Member , I want to reply to an existing open thread, so that I can contribute to ongoing discussions.	Must Have
US-10	Member	As a Member , I want to be prevented from replying to a locked thread with a clear error message, so that I understand why my action was blocked.	Must Have
US-11	Member	As a Member , I want to view and edit my profile information (full name, bio, email, avatar), so that I can keep my account details up to date.	Must Have
US-12	Member	As a Member , I want to change my password by providing my current password first, so that I can update my credentials securely.	Must Have
US-13	Member	As a Member , I want to search threads by title, message content, or author name, so that I can quickly find relevant discussions.	Must Have

US-14	Member	As a Member , I want to filter threads by category, author, tag, status, date range, and whether they contain attachments, so that I can narrow results precisely.	Should Have
US-15	Member	As a Member , I want to sort threads by newest, oldest, most viewed, or alphabetical title, so that I can prioritize results in the order most useful to me.	Should Have
US-16	Member	As a Member , I want to navigate paginated thread listings with a configurable page size, so that I can browse large volumes comfortably.	Must Have
US-17	Moderator	As a Moderator , I want to access a dashboard showing thread/message statistics and a chart of threads by category, so that I can monitor community activity at a glance.	Must Have
US-18	Moderator	As a Moderator , I want to soft-delete a message that violates community rules, so that harmful content is hidden while being preserved for audit purposes.	Must Have
US-19	Moderator	As a Moderator , I want to soft-delete an entire thread and all of its replies in one action, so that I can remove a problematic discussion efficiently.	Must Have
US-20	Admin	As an Admin , I want to view all registered users and their current roles, so that I have full visibility into who holds elevated permissions.	Must Have
US-21	Admin	As an Admin , I want to promote a regular Member to Moderator, so that I can delegate content moderation to trusted users.	Must Have
US-22	Admin	As an Admin , I want to demote a Moderator back to regular Member status, so that I can revoke privileges when no longer appropriate.	Must Have
US-23	Admin	As an Admin , I want to lock or unlock any user account (except other Admins), so that I can prevent policy-violating users from accessing the platform.	Must Have
US-24	Admin	As an Admin , I want the system to automatically soft-delete threads older than 30 days via a nightly scheduled job, so that content is managed without manual effort.	Should Have
US-25	Admin	As an Admin , I want soft-deleted content to be permanently purged after 7 days, so that storage is managed over time.	Nice to Have

3. Functional Requirements

3.1.FR-01 — Authentication and Account Management

ID	Requirement
FR-01.1	The system shall allow unauthenticated users to register by providing a unique username (3–50 characters), a valid email address, and a password (minimum 6 characters).
FR-01.2	Upon successful registration, the system shall create the account in an inactive state and issue a time-limited email verification token.
FR-01.3	The system shall activate a user account when the user visits the unique verification URL containing a valid, non-expired token.
FR-01.4	The system shall reject login attempts for unverified accounts, displaying a specific "unverified" error message distinct from a wrong-password error.
FR-01.5	The system shall reject login attempts for accounts marked inactive (banned), displaying a "disabled" error message.
FR-01.6	The system shall support persistent "Remember Me" authentication via a signed cookie valid for 30 days.
FR-01.7	The system shall invalidate the session and delete the Remember Me cookie upon explicit logout.
FR-01.8	All passwords shall be stored exclusively as BCrypt hashes; plaintext passwords shall never be persisted.

3.2.FR-02 — Thread Management

ID	Requirement
FR-02.1	Authenticated Members shall be able to create a new thread by specifying a title, a category, an initial message body, optional comma-separated tags, and an optional file attachment.
FR-02.2	The system shall support four thread statuses — OPEN, LOCKED, ARCHIVED, DELETED — enforced by a database CHECK constraint.
FR-02.3	Threads shall be soft-deleted (is_deleted = true) rather than hard-deleted when removed by a Moderator or by the automated scheduler.
FR-02.4	The system shall track a view counter per thread, incrementing it on each detail page load.
FR-02.5	Threads may be marked as sticky (is_sticky = true) to pin them to the top of listings.
FR-02.6	Tags shall be stored as a normalized many-to-many relationship; new tags shall be created automatically if they do not already exist.

3.3.FR-03 — Message (Reply) Management

ID	Requirement
----	-------------

FR-03.1	Authenticated Members shall be able to post a reply to any thread with status OPEN.
FR-03.2	The system shall reject reply submissions to threads with status other than OPEN and return a specific ThreadIsClosed error message.
FR-03.3	Each message may optionally include one file attachment; accepted types are JPG, JPEG, PNG, GIF, PDF, DOCX, and ZIP
FR-03.4	The system shall reject file attachments exceeding 5 MB, returning a specific FileTooLarge error.
FR-03.5	Uploaded files shall be stored with a UUID-prefixed filename to prevent collisions and accessible via a public URL path.
FR-03.6	Messages shall support soft-delete; soft-deleted messages shall not be displayed to regular users.
FR-03.7	When a message is updated, the system shall automatically set is_edited = true and record the updated_at timestamp via a JPA @PreUpdate hook.

3.4.FR-04 — Search and Filtering

ID	Requirement
FR-04.1	The homepage shall support keyword search across three user-selectable scopes: thread title, message content, or author name/username.
FR-04.2	The system shall support advanced filtering by any combination of: category, author, tag, thread status, date range, and presence of file attachments.
FR-04.3	The system shall perform cross-field date validation: if start date is after end date, the filter shall be disabled and an error message displayed.
FR-04.4	Thread listings shall support sorting by: newest (default), oldest, title A–Z, title Z–A, and most viewed.
FR-04.5	All thread listings shall be paginated with a configurable page size (default: 10 per page).

3.5.FR-05 — User Profile Management

ID	Requirement
FR-05.1	Authenticated Members shall be able to view their own profile page displaying username, email, full name, bio, avatar, and join date.
FR-05.2	Members shall be able to update their email, full name, bio, and avatar URL via the profile edit form.
FR-05.3	Members shall be able to change their password by first providing the correct current password; incorrect current password shall be rejected.

3.6.FR-06 — Moderation

ID	Requirement
FR-06.1	Users with ROLE_MODERATOR or ROLE_ADMIN shall have access to a Moderator dashboard at /mod/dashboard.
FR-06.2	The dashboard shall display: total thread count, total message count, the 10 most recent threads, and a bar chart of thread counts per category.
FR-06.3	Moderators shall be able to soft-delete any individual message from the dashboard.
FR-06.4	Moderators shall be able to soft-delete an entire thread along with all its child messages in a single atomic, transactional operation.

3.7.FR-07 — Administration

ID	Requirement
FR-07.1	Users with ROLE_ADMIN shall have exclusive access to the admin panel at /admin/**.
FR-07.2	The Admin shall be able to view a list of all registered users with their current roles and active status.
FR-07.3	The Admin shall be able to promote any ROLE_USER account to ROLE_MODERATOR.
FR-07.4	The Admin shall be able to demote any Moderator to ROLE_USER; the system shall prevent demotion of Admin accounts.
FR-07.5	The Admin shall be able to toggle the active status (lock/unlock) of any non-Admin account.

3.8.FR-08 — Automated Retention Policy

ID	Requirement
FR-08.1	A scheduled job shall run nightly at 00:00 and soft-delete all threads (and their messages) created more than 30 days ago that are not already deleted.
FR-08.2	A second nightly job shall permanently hard-delete all threads that have been in the soft-deleted state for more than 7 days.

4. Non-Functional Requirements

4.1.Performance

ID	Requirement
----	-------------

NFR-P1	The thread listing homepage (default 10 items, no filters) shall load within 2 seconds under normal operating conditions.
NFR-P2	Thread detail pages shall load within 1.5 seconds for threads with up to 100 replies.
NFR-P3	Search and filter queries shall return results within 3 seconds for datasets up to 10,000 threads, supported by the composite index (category_id, status, created_at) and full-text index on threads.title defined in the schema.
NFR-P4	The system shall support a minimum of 50 concurrent users without degradation, utilizing Spring Boot's embedded Tomcat thread pool.
NFR-P5	File uploads shall be processed and stored within 5 seconds for attachments up to the enforced 5 MB limit.

4.2.Security

ID	Requirement
NFR-S1	All passwords shall be hashed using BCrypt via Spring Security's BCryptPasswordEncoder; plaintext passwords shall never appear in logs, the database, or responses.
NFR-S2	Session management shall use Spring Security's session-based mechanism; sessions shall be invalidated on logout and the JSESSIONID cookie deleted.
NFR-S3	All database interactions shall use parameterized queries via Spring Data JPA and Hibernate ORM, fully preventing SQL injection.
NFR-S4	All HTML output shall be escaped by Thymeleaf's default rendering engine, preventing reflected and stored XSS attacks.
NFR-S5	URL-level access control shall be enforced by Spring Security: /mod/** requires ROLE_MODERATOR or ROLE_ADMIN; /admin/** requires ROLE_ADMIN exclusively.
NFR-S6	Email verification tokens shall have a defined expiry date; expired tokens shall be rejected and the account shall remain inactive.
NFR-S7	Admin accounts shall be protected from self-demotion and self-locking by explicit server-side role checks before any status change is persisted.

4.3.Usability

ID	Requirement
NFR-U1	The application shall be fully responsive across desktop ($\geq 1024\text{px}$), tablet ($768\text{--}1023\text{px}$), and mobile ($\leq 767\text{px}$) screen widths, implemented via Bootstrap's responsive grid.
NFR-U2	All form validation errors shall be displayed inline, adjacent to the offending field, with specific human-readable messages (e.g., <i>"Username must be between 3 and 50 characters"</i>).
NFR-U3	Error states (locked thread, invalid file type, expired verification token) shall display specific, actionable messages rather than generic error pages.
NFR-U4	The interface shall provide a light/dark theme toggle (implemented in theme.js) that persists across page reloads.

4.4. Scalability

ID	Requirement
NFR-SC1	The stateless MVC architecture (Spring Boot + Thymeleaf + MySQL) shall allow horizontal scaling by adding application server instances behind a load balancer without code changes.
NFR-SC2	All foreign keys and frequently queried columns shall be indexed to maintain query performance as data volume grows toward 100,000+ records.
NFR-SC3	The automated two-stage retention policy (30-day soft-delete → 7-day hard-delete) shall act as a built-in data lifecycle mechanism, preventing unbounded table growth.
NFR-SC4	File uploads shall be stored on the local filesystem at a configurable path, making migration to external object storage (e.g., AWS S3) feasible without architectural changes.

4.5. Reliability

ID	Requirement
NFR-R1	The application shall target 99% uptime during the evaluation period, with global exception handling preventing unhandled errors from producing white-screen failures.
NFR-R2	All multi-step database operations (user registration with token creation, thread deletion with cascading message deletion) shall be wrapped in <code>@Transactional</code> boundaries to ensure atomicity.
NFR-R3	Application logs shall be written to a rolling log file (logs/forum-app.log) with 30-day retention, capturing scheduled job execution and error events at appropriate SLF4J log levels
NFR-R4	The database schema shall enforce referential integrity via foreign key constraints with explicit ON DELETE rules — CASCADE for junction tables (thread_tags, thread_likes), RESTRICT for category-to-thread relationships — preventing orphaned records.
NFR-R5	File size limits shall be enforced at two layers: application logic (5 MB per message in HomeController) and server configuration (spring.servlet.multipart.max-file-size=10MB), providing defense-in-depth.

III. System Architecture

1. Architecture Overview

The IU Forum application follows a classic **three-tier architecture** composed of a presentation tier (browser + Thymeleaf templates), an application tier (Spring Boot MVC), and a data tier (MySQL 8.0). All HTML is rendered server-side: the browser sends HTTP

requests, the Spring MVC layer processes them, queries the database via JPA, and returns fully-formed HTML pages. There is no client-side rendering framework; JavaScript is used only for progressive enhancement (theme toggling and inline form validation), keeping the application functional even with JavaScript disabled.

Spring Security acts as a cross-cutting concern, intercepting every inbound request through a filter chain before it reaches any controller. This ensures that authentication and authorization are enforced uniformly at the infrastructure level, independent of individual controller logic. A secondary layer of business-level guards (e.g., `principal == null` checks in `ThreadController`) provides defense-in-depth for sensitive write operations.

File attachments represent a fourth concern outside the three tiers: uploaded files are stored on the server's local filesystem under `/uploads/`, with their public URL paths persisted in the `messages.file_url` column. This keeps binary data out of the database while remaining accessible via Spring's static resource handler.

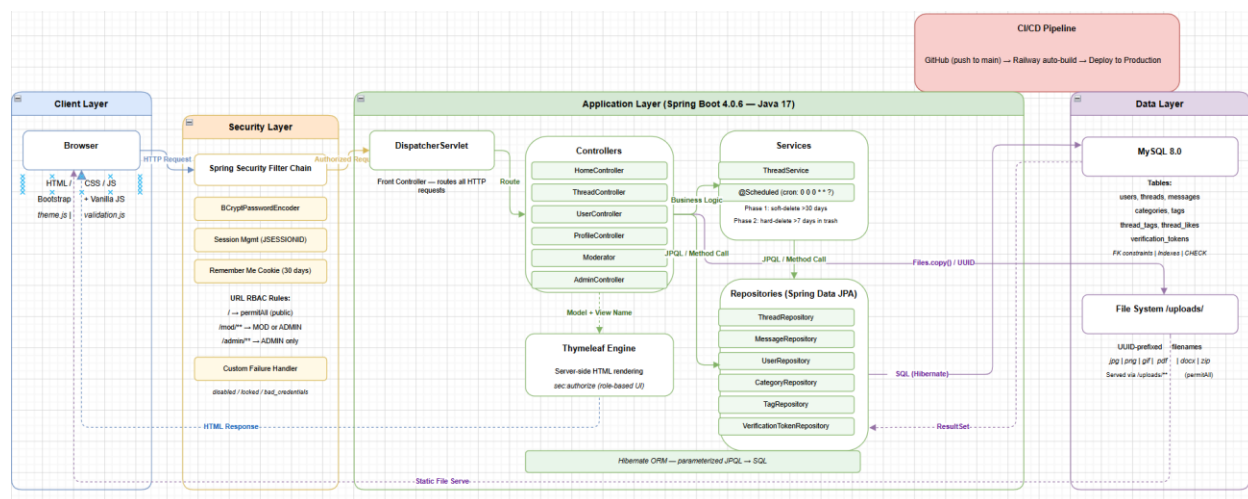


Figure 3.1: architecture diagram

2. Technology Stack Justification

2.1. Frontend — Thymeleaf + Vanilla JavaScript vs. React / Vue.js

Thymeleaf is a Java-native server-side template engine that integrates directly with Spring MVC's Model object. When a controller populates the model with a `Page<Thread>`, Thymeleaf iterates over it in the template with `th:each` and renders complete HTML before the response leaves the server. This approach was chosen over a Single-Page Application (SPA) framework such as React or Vue.js for three concrete reasons. First, it eliminates the need for a separate REST API layer — controllers return view names, not JSON, removing an entire layer of code. Second, Thymeleaf's Spring Security dialect

(`sec:authorize`) allows role-based UI elements (e.g., hiding the "Delete" button from non-moderators) to be declared declaratively in HTML, enforced at render time on the server rather than in client-side JavaScript that can be manipulated. Third, the team's existing Java proficiency meant Thymeleaf's Java-aligned syntax had a significantly lower learning curve.

Vanilla JavaScript was chosen for the two interactive behaviors that Thymeleaf alone cannot handle: theme persistence (`theme.js`, using `localStorage`) and client-side form validation (`validation.js`). A full JS framework such as `Vue.js` was considered unnecessary for this limited scope of interactivity and would have introduced a build toolchain (`npm`, `Webpack/Vite`) adding complexity without proportionate benefit.

Trade-off: The server-rendered model means every navigation requires a full-page HTTP round trip. SPAs deliver a more fluid user experience through partial DOM updates, which this application sacrifices in exchange for architectural simplicity and tighter security control.

2.2.Backend — Spring Boot 4.0.6 vs. Raw Java Servlets

Spring Boot was chosen as the backend framework because it eliminates the extensive boilerplate configuration required by raw Java Servlets and Jakarta EE. The specific features that would have required manual implementation with raw Servlets include:

- **Auto-configuration:** database connection pooling, JPA entity scanning, and Tomcat embedding are configured automatically from `application.properties`.
- **Spring Security:** BCrypt password hashing, session management, Remember Me cookies, and URL-pattern RBAC would each require hundreds of lines of custom filter and listener code with raw Servlets.
- **Spring Data JPA:** the six repository interfaces are implemented at runtime by Spring; a raw Servlet project would require explicit DAO classes with JDBC `PreparedStatement` boilerplate for every query.
- **@Scheduled:** the two nightly retention jobs in `ThreadService` are declared with a single annotation; raw Servlets would require a `ScheduledExecutorService` or an external scheduler like Quartz.
- **@Transactional:** atomic multi-step database operations (e.g., soft-deleting a thread and all its messages) are guaranteed by a single annotation; raw code would require explicit `connection.setAutoCommit(false)` and manual rollback logic.

Trade-off: Spring Boot's abstraction introduces a heavier application footprint and a steeper debugging curve when auto-configuration behaves unexpectedly. A developer

unfamiliar with Spring's dependency injection container may struggle to trace why a bean is wired incorrectly. For a small team on a time-limited project, however, the productivity gains significantly outweighed this cost.

2.3.Database — MySQL 8.0 vs. MongoDB

The forum's data is inherently relational. Users author Threads; Threads contain Messages; Threads belong to Categories; Threads and Tags share a many-to-many relationship via the `thread_tags` junction table; Users and Threads share a many-to-many relationship via `thread_likes`. These normalized relationships are a natural fit for a relational database, where foreign key constraints (`fk_threads_category`, cascade rules on `thread_tags`) enforce referential integrity at the storage layer.

MongoDB's document model would require embedding messages inside thread documents or managing manual references between collections. As thread message counts grow, embedded documents create unbounded document growth; separate collections lose the join semantics that make queries like "find all threads by this author in this category with these tags" straightforward in SQL. The advancedFilter JPQL query in `ThreadRepository`, which joins across threads, categories, tags, messages, and users in a single query, exemplifies logic that would require multiple round trips or aggregation pipelines in MongoDB.

Additionally, MySQL 8.0's native full-text index on `threads.title` (`idx_threads_title_ft`) supports the keyword search feature without any external search infrastructure.

Trade-off: MySQL requires a predefined schema, making structural changes (adding a column, altering a relationship) more deliberate than MongoDB's schema-less flexibility. For a domain as well-defined as a forum, this rigidity is an asset rather than a liability.

2.4.ORM — Spring Data JPA (Hibernate) vs. Raw JDBC

Spring Data JPA provides two distinct layers of value. First, the repository interface naming convention (e.g., `findByTitleContainingIgnoreCaseAndDeletedFalse`) generates complete JPQL queries at startup, eliminating repetitive `PreparedStatement` construction. Second, the `@Query` annotation on `ThreadRepository.advancedFilter()` allows complex multi-join queries with named parameters, providing full SQL control where needed while remaining within the JPA abstraction.

Additional benefits realized in this project: `@Transactional` on `Moderator.deleteThread()` guarantees that soft-deleting the parent thread and all child

messages either both succeed or both roll back; @PreUpdate lifecycle hooks on Message and Thread automatically maintain updated_at and is_edited without any controller code.

Trade-off: Hibernate's lazy loading introduces the risk of N+1 query problems — loading a list of threads and then accessing each thread's messages in a Thymeleaf template can trigger one SQL query per thread. This risk must be managed through careful use of JOIN FETCH or DTO projections, which requires developers to understand Hibernate's session lifecycle.

2.5.Authentication — Spring Security vs. Custom Session Management

Spring Security was chosen because it provides a production-grade security infrastructure as a single dependency. Features used directly in this project: BCryptPasswordEncoder (adaptive hashing with configurable work factor), URL-pattern RBAC configured in SecurityConfig.securityFilterChain(), form-login with custom failure handler that distinguishes between wrong-password, disabled-account, and locked-account errors, Remember Me via a signed HMAC cookie valid for 30 days, and automatic session invalidation plus cookie deletion on logout.

Implementing equivalent functionality from scratch — a correct BCrypt implementation, a CSRF-safe session store, a signed persistent cookie, and URL interception — would represent a significant security risk. Security code written under time pressure by developers without a security specialization is a common source of vulnerabilities.

Trade-off: Spring Security's configuration API is notoriously verbose and the mental model for its filter chain ordering has a steep learning curve. In this project, CSRF protection was disabled (csrf.disable()) to simplify form submissions in the development/lab environment — a trade-off that would be unacceptable in a production deployment and should be re-enabled with proper token handling before public release.

2.6.Deployment — Railway vs. Heroku / AWS EC2

Railway was selected as the hosting platform for four practical reasons. First, it integrates directly with GitHub: pushing to the main branch triggers an automatic build and deploy without configuring a separate CI/CD tool. Second, Railway provides a MySQL addon that provisions and links a production database to the application via an environment variable, eliminating manual database server setup. Third, Railway's free tier has no "cold start" delay, unlike Heroku's free dynos which sleep after 30 minutes of inactivity and impose a 10–30 second wake-up penalty during demos. Fourth, Railway

supports environment variable injection, keeping credentials (DB_PASSWORD, SPRING_DATASOURCE_URL) out of the source repository.

Trade-off: Railway represents vendor lock-in: the deployment configuration, environment variable names, and MySQL addon are Railway-specific. Migrating to AWS EC2 or Google Cloud Run would require reconfiguring the deployment pipeline. A self-managed VPS would offer more control over OS-level tuning, custom MySQL configuration, and resource allocation, at the cost of DevOps overhead that is disproportionate for an academic project.

3. Design Patterns

3.1. Model-View-Controller (MVC)

MVC is the foundational architectural pattern of the application, enforced by the Spring MVC framework. The three roles are clearly separated:

- **Model:** JPA entity classes (User, Thread, Message, Category, Tag, VerificationToken) represent the domain data. They are plain Java objects annotated with `@Entity` and contain no view or routing logic.
- **View:** Thymeleaf HTML templates (common/index.html, common/thread-detail.html, mod/dashboard.html, admin/manage-mods.html, etc.) are responsible solely for rendering data into HTML. They reference model attributes via th:expressions but contain no business logic.
- **Controller:** Six `@Controller` classes (HomeController, ThreadController, UserController, ProfileController, Moderator, AdminController) receive HTTP requests, invoke repositories or services, populate the Model object, and return a view name string.

Spring's DispatcherServlet acts as the MVC front controller: it receives every request, consults HandlerMapping to identify the correct controller method, invokes it, receives the ModelAndView result, and delegates rendering to Thymeleaf's ViewResolver. This central dispatcher is what makes Spring's MVC implementation a clean realization of the pattern.

3.2. Repository Pattern

The Repository pattern abstracts the data access layer behind domain-oriented interfaces, decoupling business logic from persistence technology. In this project, six interfaces extend `JpaRepository<Entity, Long>`: ThreadRepository, MessageRepository, UserRepository, CategoryRepository, TagRepository, and VerificationTokenRepository.

Controllers and services interact exclusively with these interfaces — they never instantiate a connection, write SQL, or manage transactions manually. Spring Data JPA generates concrete proxy implementations at application startup, translating method signatures into JPQL (e.g., `findByDeletedFalse(Pageable)` → `SELECT t FROM Thread t WHERE t.deleted = false`) and custom `@Query` annotations into parameterized SQL.

The practical benefit is testability and replaceability: the repository interface could be backed by a different database (H2 for tests, PostgreSQL in production) by changing one configuration property, with no changes to controller or service code.

3.3.Singleton Pattern

Spring's default bean scope is singleton: every class annotated with `@Component`, `@Service`, `@Repository`, `@Controller`, or `@Configuration` is instantiated once and shared across the entire application. This is Spring's implementation of the Singleton pattern, managed by the `ApplicationContext` container rather than by individual classes.

Two singletons are particularly significant in this codebase. `SecurityConfig` is instantiated once and provides the single `PasswordEncoder` bean (BCrypt) that is injected into `UserController` and `ProfileController`. Because it is a singleton, all password hashing operations share the same encoder instance with consistent configuration. `ThreadService` is the singleton that owns the two `@Scheduled` nightly jobs — having a single instance guarantees that the retention jobs are not duplicated across multiple threads.

3.4.Soft Delete Pattern

The Soft Delete pattern treats deletion as a state change rather than a destructive operation. In this codebase, the `Thread`, `Message`, and `Category` entities each contain an `is_deleted` bit column (mapped to a boolean `deleted` field). When a `Moderator` deletes a message via `/mod/delete-message/{id}`, the `Moderator` controller sets `msg.setDeleted(true)` and saves — the row remains in the database but is invisible to all queries that include `AND deleted = false`.

This pattern implements a two-phase lifecycle enforced by `ThreadService`:

1. Threads active for more than 30 days are soft-deleted by `autoDeleteOldThreads()` (runs nightly at 00:00 via `@Scheduled(cron = "0 0 0 * * ?")`).
2. Threads in the soft-deleted state for more than 7 days are hard-deleted by `autoDeleteOldSoftDeletedThreads()`.

The value of this pattern is threefold: it provides a recovery window for accidental deletions, it maintains an audit trail (the record still exists with its `created_at` timestamp), and it enforces moderator accountability — removed content is not immediately destroyed.

3.5.Template Method Pattern (JPA Lifecycle Hooks)

The Template Method pattern defines a skeleton algorithm in a base class and allows subclasses to override specific steps. JPA's `@PreUpdate` lifecycle callback is an implicit application of this pattern: the JPA framework (Hibernate) defines the algorithm for persisting entity updates and provides a hook — the method annotated `@PreUpdate` — that the entity class implements to inject custom behavior before the SQL UPDATE is issued.

In `Message.onUpdate()`, the hook automatically sets `this.updatedAt = LocalDateTime.now()` and `this.edited = true` every time a message is updated, regardless of which controller triggered the update. In `Thread.onUpdate()`, the same hook maintains `updatedAt`. This ensures these audit fields are always correctly maintained without requiring any controller to remember to set them explicitly — the framework calls the hook; the entity defines what happens.

3.6.Factory Pattern (Spring IoC Container)

Spring's `ApplicationContext` functions as a Factory for all managed beans. Rather than controllers instantiating their dependencies with `new`, they declare dependencies via `@Autowired`, and Spring's IoC container constructs and injects the correct instance.

The most explicit Factory-style configuration in this codebase is in `SecurityConfig`, where the `@Bean`-annotated `passwordEncoder()` method acts as a factory method: it constructs and returns a `BCryptPasswordEncoder` instance that the container then injects wherever a `PasswordEncoder` is declared as a dependency. `UserController` and `ProfileController` both receive this exact instance, ensuring consistent hashing behavior across all password operations without either class knowing the concrete type or construction parameters of the encoder.

4. API Design

The forum system implements a RESTful API architecture to ensure scalability, maintainability, and a strict separation of concerns between the user interface (Frontend) and the business logic (Backend).

4.1 RESTful principles followed

The system strictly adheres to core REST (Representational State Transfer) principles:

- **Client-Server Architecture:** Complete separation between the UI presentation and database management.
- **Statelessness:** Each request from the client to the server contains all the necessary information to understand and process it. The system uses JWT Tokens in the Request Header for authentication instead of relying on server-side sessions.
- **Resource-Oriented:** URLs are structured using plural nouns representing entities (e.g., threads, users), strictly avoiding verbs in the endpoint paths.
- **Standardized Responses:** All returned data is consistently formatted in JSON.

4.2 URL structure

The URL structure is designed to be consistent and includes versioning to facilitate future API upgrades without breaking legacy client applications.

- **Base URL:** /api/v1
- **Naming Convention:** /api/v1/{resources}/{id}
- **Nested Resources:** Used to represent parent-child relationships. For example, to retrieve comments for a specific thread:
/api/v1/threads/{threadId}/comments

4.3 HTTP methods usage

The system utilizes standard HTTP methods to define CRUD (Create, Read, Update, Delete) actions on resources:

- **GET:** Retrieve and read resources (e.g., fetch a list of threads, view thread details). These requests are safe and idempotent (do not alter data).
- **POST:** Create a new resource (e.g., register an account, publish a new thread, submit a comment).
- **PUT:** Update an existing resource entirely (e.g., edit thread content).
- **DELETE:** Remove a resource. In this project, the DELETE method triggers a **Soft Delete** mechanism (updating the `is_deleted = true` flag in the database) rather than a Hard Delete, in order to preserve data integrity and audit trails.

4.4 Status codes strategy

The API returns standard HTTP status codes, allowing the Client side to easily handle errors and display appropriate user feedback:

- **200 OK:** The request succeeded (typically used for GET, PUT, and DELETE operations).

- **201 Created:** A new resource was successfully created (used for POST operations).
- **400 Bad Request:** The request was invalid (e.g., missing parameters, invalid email format).
- **401 Unauthorized:** The request requires authentication, but the Client provided no token or an expired token.
- **403 Forbidden:** The user is authenticated but lacks the necessary permissions to perform the action (e.g., a standard User attempting to access Admin-only APIs).
- **404 Not Found:** The requested resource does not exist (or has been Soft Deleted).
- **500 Internal Server Error:** An unexpected error occurred on the server or database side.

4.5 Request/response examples for key endpoints

Below is the data flow illustration for one of the system's core endpoints: **Create a new thread**.

Endpoint: POST /api/v1/threads **Description:** Publish a new thread to the forum. Requires user authentication (Bearer Token provided).

Request Headers:

```
Authorization: Bearer <jwt_token_here>
Content-Type: application/json
```

Request Body (JSON):

```
{
  "title": "MySQL Query Optimization Guide",
  "content": "I have a large data table, what is the most effective way to implement indexing?",
  "category_id": 2
}
```

Response (201 Created):

```
{
  "success": true,
  "message": "Thread created successfully",
  "data": {
    "id": 105,
    "title": "MySQL Query Optimization Guide",
    "content": "I have a large data table, what is the most effective way to implement indexing?",
    "category": {
      "id": 2,
```

```
    "name": "Database"
  },
  "author": {
    "id": 15,
    "username": "student_coder"
  },
  "created_at": "2026-06-03T14:30:00Z",
  "is_deleted": false
}
```

IV. Database Design

1. Entity-Relationship Diagram

The database schema consists of **8 tables** representing the core domain of an academic discussion forum. The database design uses a relational schema because the application contains clear relationships between users, categories, threads, messages, tags, thread_tag, thread_like, verification_tokens. Junction tables are used for many-to-many relationships, and foreign keys enforce referential integrity.

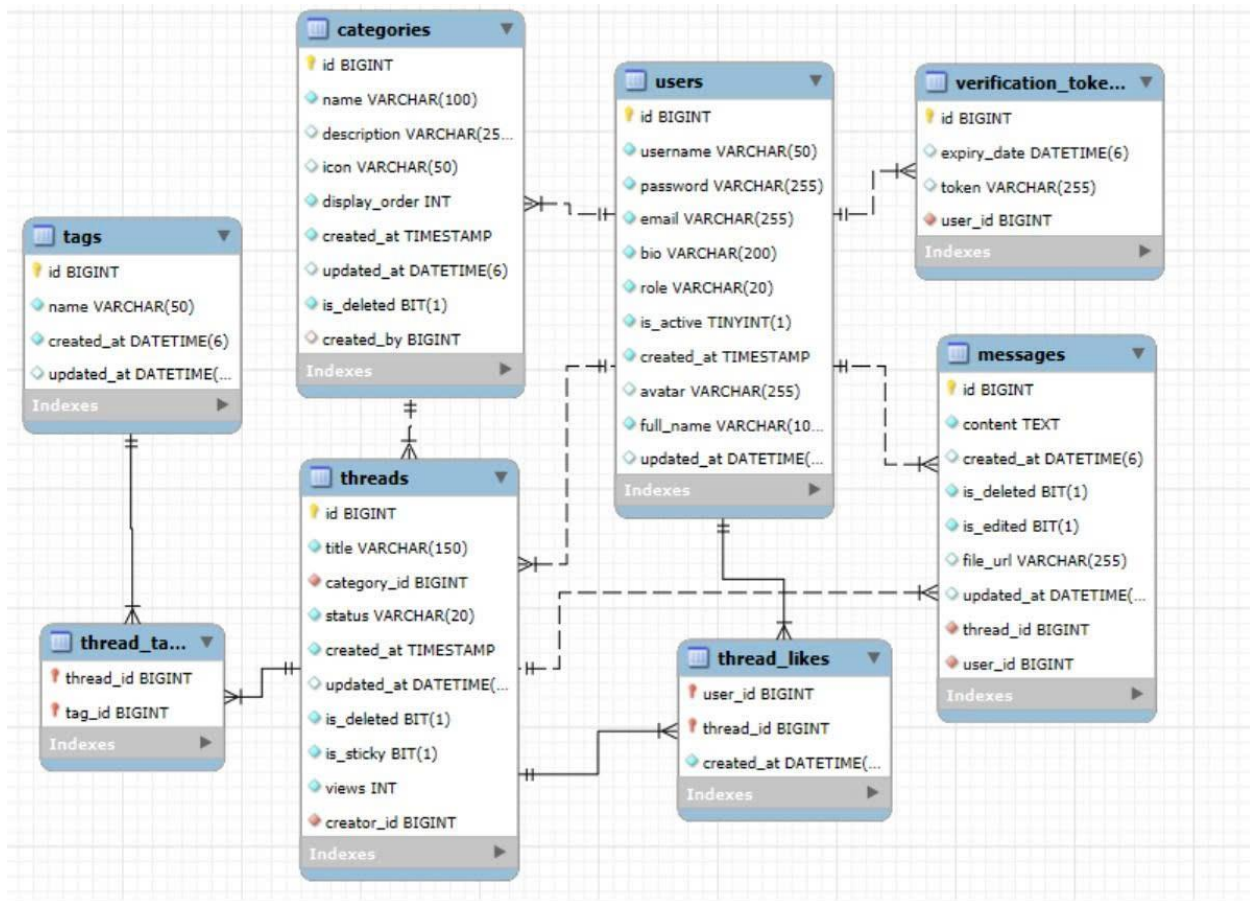


Figure 4.1: Entity – Relationship Diagram

Relationship Summary:

Relationship	Types	Foreign Key Location
users → categories	One-to-Many	categories.created_by
users → threads	One-to-Many	threads.creator_id
users → messages	One-to-Many	messages.user_id
categories → threads	One-to-Many	threads.category_id
threads → messages	One-to-Many	messages.thread_id
threads ↔ tags	Many-to-Many	Junction: thread_tags
users ↔ threads	Many-to-Many	Junction: thread_likes
users → verification_tokens	One-to-Many	verification_tokens.user_id

2. Table Schemas

2.1. TABLE: users

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Surrogate primary key
username	VARCHAR(50)	NOT NULL, UNIQUE	Login identifier, must be unique
password	VARCHAR(255)	NOT NULL	BCrypt-hashed password
email	VARCHAR(255)	NOT NULL, UNIQUE	User email must be unique
full_name	VARCHAR(100)	NOT NULL	Display name of the user
bio	VARCHAR(200)	NOT NULL	Short self-introduction
avatar	VARCHAR(255)	NULL	URL path to profile picture
role	VARCHAR(20)	NOT NULL	Access role: ROLE_USER / ROLE_MODERATOR / ROLE_ADMIN
is_active	TINYINT(1)	NOT NULL, DEFAULT 1	0 = banned or deactivated account
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Account creation timestamp
updated_at	DATETIME(6)	NULL	Last profile update timestamp

2.2. TABLE: categories

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Surrogate primary key
name	VARCHAR(100)	NOT NULL, UNIQUE	Category display name
description	VARCHAR(255)	NULL	Brief description of the category
icon	VARCHAR(50)	NULL	Emoji or icon identifier for UI display
display_order	INT	NOT NULL, DEFAULT 0	Controls the rendering order in the navigation menu
is_deleted	BIT(1)	NOT NULL	Soft-delete flag

created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Creation timestamp
updated_at	DATETIME(6)	NULL	Last modification timestamp
created_by	BIGINT	FK → users(id), ON DELETE SET NULL	ID of the admin who created the category

2.3. TABLE: threads

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Surrogate primary key
title	VARCHAR(150)	NOT NULL	Thread title, indexed with FULLTEXT
category_id	BIGINT	NOT NULL, FK → categories(id), ON DELETE RESTRICT	Category this thread belongs to
creator_id	BIGINT	NOT NULL, FK → users(id)	Author of the thread
status	VARCHAR(20)	NOT NULL, CHECK IN ('OPEN', 'LOCKED', 'ARCHIVED', 'DELETED')	Current lifecycle state of the thread
is_sticky	BIT(1)	NOT NULL	If true, thread is pinned to the top of the category
is_deleted	BIT(1)	NOT NULL	Soft-delete flag
views	INT	NOT NULL	Incremented each time the thread is viewed
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Thread creation timestamp
updated_at	DATETIME(6)	NULL	Last update timestamp

2.4. TABLE: messages

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Surrogate primary key
content	TEXT	NOT NULL	Full body text of the reply
thread_id	BIGINT	NOT NULL, FK → threads(id)	Thread this message belongs to
user_id	BIGINT	NOT NULL, FK → users(id)	Author of the message

file_url	VARCHAR(255)	NULL	URL of an attached file, if any
is_edited	BIT(1)	NOT NULL	Flags whether the message has been edited after posting
is_deleted	BIT(1)	NOT NULL	Soft-delete flag
created_at	DATETIME(6)	NULL	Message creation timestamp
updated_at	DATETIME(6)	NULL	Last edit timestamp

2.5. TABLE: tags

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Surrogate primary key
name	VARCHAR(50)	NOT NULL, UNIQUE	Tag label (e.g., "Java", "IELTS")
created_at	DATETIME(6)	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Tag creation timestamp
updated_at	DATETIME(6)	NULL	Last update timestamp

2.6. TABLE: thread_tags (Junction)

Column Name	Data Type	Constraints	Description
thread_id	BIGINT	PK (composite), FK → threads(id), ON DELETE CASCADE	Reference to the thread
tag_id	BIGINT	PK (composite), FK → tags(id), ON DELETE CASCADE	Reference to the tag

2.7. TABLE: thread_likes (Junction)

Column Name	Data Type	Constraints	Description
user_id	BIGINT	PK (composite), FK → users(id), ON DELETE CASCADE	The user who liked the thread
thread_id	BIGINT	PK (composite), FK → threads(id), ON DELETE CASCADE	The thread that was liked
created_at	DATETIME(6)	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp of the like action

2.8. TABLE: verification_tokens

Column Name	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Surrogate primary key
token	VARCHAR(255)	UNIQUE	Randomly generated UUID token sent via email
expiry_date	DATETIME(6)	NULL	Token expiration deadline
user_id	BIGINT	NOT NULL, UNIQUE, FK → users(id)	The user this token was issued for (one token per user)

3. Relationships Explanation

Relationship	Explanation
Users → Categories (1:N)	A user can create multiple categories, while each category records only one creator through the created_by foreign key. This relationship represents administrative ownership of forum categories. The foreign key is stored in the categories table. The relationship uses ON DELETE SET NULL , meaning that if the creator account is removed, the category remains available and only the creator reference is cleared. This prevents the loss of forum structure when an administrator leaves the system.
Users → Threads (1:N)	A user can create many discussion threads, while each thread has exactly one creator identified by creator_id. This relationship represents thread authorship and supports features such as ownership tracking, moderation, and user activity statistics. The foreign key is stored in the threads table.
Users → Messages (1:N)	A user can publish multiple messages across different discussion threads, while each message belongs to a single author. The relationship is implemented through the user_id foreign key in the messages table. This design preserves message ownership and allows the system to display author information for every post.
Categories → Threads (1:N)	A category can contain many threads, while each thread belongs to exactly one category. The relationship is implemented through the category_id foreign key in the threads table. The relationship uses ON DELETE RESTRICT , which prevents a category from being deleted while threads still reference it. This avoids orphaned discussions and maintains a consistent forum hierarchy. A thread acts as a container for discussion and can contain many messages. Each message belongs to only one thread through the thread_id foreign key.

	This relationship reflects the hierarchical structure of forum discussions, where replies are grouped under a specific topic.
Threads → Messages (1:N)	A thread acts as a container for discussion and can contain many messages. Each message belongs to only one thread through the thread_id foreign key. This relationship reflects the hierarchical structure of forum discussions, where replies are grouped under a specific topic.
Threads ↔ Tags (M:N) via thread_tags	A thread can have multiple tags, and a tag can be assigned to multiple threads. This many-to-many relationship is implemented using the thread_tags junction table, which contains thread_id and tag_id. The design allows flexible categorization without duplicating tag information. The relationship uses ON DELETE CASCADE , ensuring that association records are automatically removed when either the related thread or tag is deleted.
Users ↔ Threads (M:N) via thread_likes	Users can like multiple threads, and each thread can receive likes from multiple users. This many-to-many relationship is implemented through the thread_likes junction table. In addition to the foreign keys (user_id and thread_id), the table stores created_at, which records when the like occurred. The relationship uses ON DELETE CASCADE on thread_id, ensuring that likes are automatically removed when a thread is deleted.
Users → Verification_Tokens (1:N)	A user may have multiple verification tokens over time, while each token belongs to exactly one user. The relationship is implemented through the user_id foreign key in the verification_tokens table. This design supports account verification and token regeneration when previous tokens expire or become invalid.

4. Normalization Analysis

The IU Forum database schema is designed to comply with **Third Normal Form (3NF)**. The following analysis demonstrates compliance at each level of normalization.

- **First Normal Form (1NF)**: A relation is in 1NF if all column values are atomic (indivisible) and each column contains values of a single type. The schema satisfies 1NF throughout: every column stores a single scalar value, and no column contains repeating groups or array-type data. For example, rather than storing multiple tags as a comma-separated string within the threads table, the schema correctly externalizes the multi-valued attribute into a separate tags table and resolves the relationship through the thread_tags junction table.

- **Second Normal Form (2NF)**: A relation is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the entire primary key (no partial

dependencies). For tables with a single-column surrogate primary key — such as users, threads, messages, categories, and tags — 2NF is trivially satisfied. For the junction tables thread_tags and thread_likes, which use composite primary keys, there are no non-key attributes that depend on only one part of the composite key (thread_likes.created_at depends on the full composite key (user_id, thread_id), not on either column alone), confirming 2NF compliance.

- **Third Normal Form (3NF):** A relation is in 3NF if it is in 2NF and no non-key attribute is transitively dependent on the primary key through another non-key attribute. The threads table is used here as a representative example.

Table	Functional Dependency
users	id → username, password, email, full_name, bio, avatar, role, is_active, created_at, updated_at
categories	id → name, description, icon, display_order, is_deleted, created_at, updated_at, created_by
threads	id → title, category_id, creator_id, status, is_sticky, is_deleted, views, created_at, updated_at
messages	id → content, thread_id, user_id, file_url, is_edited, is_deleted, created_at, updated_at
tags	id → name, created_at, updated_at
thread_tags	(thread_id, tag_id) → ∅ (no non-key attributes)
thread_likes	(user_id, thread_id) → created_at
Verification_tokens	id → token, expiry_date, user_id

One instance of deliberate denormalization exists in the schema: the threads.views column stores a running count of how many times a thread has been viewed. In a fully normalized design, this value could theoretically be derived by counting access log records. However, maintaining a separate log table and performing a COUNT(*) aggregation on every thread listing page request would impose unacceptable query overhead at scale. Storing views as a pre-computed counter directly in threads is a standard and widely accepted performance optimization, and this design decision is acknowledged as intentional denormalization justified by read performance requirements.

5. Indexing Strategy

Index Inventory:

Index Name	Table	Columns	Type	Purpose
PRIMARY	All tables	id	B-Tree (implicit)	Unique row identification and clustered access
uq_users_username	users	username	Unique B-Tree	Enforces uniqueness; speeds up login lookups by username
uq_users_email	users	email	Unique B-Tree	Enforces uniqueness; speeds up duplicate-check on registration
idx_users_role	users	role	B-Tree	Speeds up admin queries filtering users by role (e.g., list all moderators)
idx_users_is_active	users	is_active	B-Tree	Speeds up filtering active vs. banned accounts in user management
idx_categories_display_order	categories	display_order	B-Tree	Speeds up ORDER BY display_order when rendering the navigation menu
idx_categories_created_by	categories	created_by	B-Tree	Required index on FK column; speeds up JOIN to find categories by creator
idx_threads_category_created	threads	(category_id, created_at DESC)	Composite B-Tree	Optimizes the most common listing query: threads in a category ordered by recency
idx_threads_status	threads	status	B-Tree	Speeds up filtering by thread lifecycle state (e.g., showing only OPEN threads)
idx_threads_cat_status_created	threads	(category_id, status, created_at DESC)	Composite B-Tree	Optimizes filtered listing: active threads in a category ordered by date

idx_threads_title_ft	threads	title	FULLTEXT	Enables efficient keyword search across thread titles using MATCH ... AGAINST
idx_thread_tags_tag	thread_tags	tag_id	B-Tree	Speeds up the reverse lookup: find all threads associated with a given tag
idx_thread_likes_thread	thread_likes	thread_id	B-Tree	Speeds up counting or retrieving all likes for a specific thread

V. Implementation Details

1. Key Algorithms & Business Logic

Algorithm 1 — Multi-criteria Advanced Filter

The most technically complex query in the system is ThreadRepository.advancedFilter(), a single JPQL @Query that accepts seven independent nullable filter parameters: categoryId, authorId, tagId, status, startDate, endDate, and hasImage. The challenge is that any combination of these parameters may be present or absent at runtime — naively, this would require $2^7 = 128$ distinct query methods. The solution uses a short-circuit IS NULL pattern for each parameter:

```
@Query("SELECT DISTINCT t FROM Thread t " +
    "LEFT JOIN t.tags tag " +
    "WHERE t.deleted = false " +
    "AND (:categoryId IS NULL OR t.category.id = :categoryId) " +
    "AND (:tagId IS NULL OR tag.id = :tagId) " +
    "AND (:status IS NULL OR :status = '' OR t.status = :status) " +
    "AND (CAST(:startDate AS date) IS NULL OR t.createdAt >= :startDate) " +
    "AND (:hasImage IS NULL OR :hasImage = false " +
    "    OR EXISTS (SELECT 1 FROM Message m WHERE m.thread = t " +
    "        AND m.fileUrl IS NOT NULL AND m.fileUrl != ''))")
Page<Thread> advancedFilter(@Param("categoryId") Long categoryId, ...);
```

Three design decisions make this non-trivial. First, the LEFT JOIN t.tags is required for many-to-many tag filtering — an inner join would exclude threads with no tags. DISTINCT prevents duplicate rows when a thread has multiple tags matching the join. Second, the EXISTS subquery for hasImage avoids joining the messages table into the main result set, which would multiply rows. Third, before this query is ever called,

HomeController.index() performs cross-field validation: if startDate.isAfter(endDate), both are nullified and a dateError attribute is added to the model, ensuring the database never receives an invalid date range.

Algorithm 2 — Two-Phase Content Retention Policy

ThreadService implements an automated content lifecycle using two @Scheduled methods, both triggered by the cron expression "0 0 0 * * ?" (midnight daily).

Phase 1 — Soft-delete (autoDeleteOldThreads): threads created more than 30 days ago where deleted = false are located via threadRepository.findByCreatedAtBeforeAndDeletedFalse(now().minusDays(30)). Each thread has setDeleted(true) applied, and the same flag is cascaded to every child message via a loop and messageRepository.saveAll().

Phase 2 — Hard-delete (autoDeleteOldSoftDeletedThreads): threads where deleted = true and createdAt is more than 7 days old are permanently removed via threadRepository.deleteAll().

```
@Scheduled(cron = "0 0 0 * * ?")
@Transactional
public void autoDeleteOldThreads() {
    LocalDateTime threshold = LocalDateTime.now().minusDays(30);
    List<Thread> old = threadRepository
        .findByCreatedAtBeforeAndDeletedFalse(threshold);
    for (Thread t : old) {
        t.setDeleted(true);
        messageRepository.findByThread(t)
            .forEach(m -> m.setDeleted(true));
    }
    threadRepository.saveAll(old);
    log.info("Đã ẨN {} bài viết cũ hơn 30 ngày.", old.size());
}
```

The two-phase design is deliberate: Phase 1 gives moderators a 7-day recovery window after automated soft-deletion. The @Transactional annotation on both methods ensures that if the message cascade fails mid-loop, the entire operation rolls back to a consistent state. SLF4J parameterized logging (log.info("... {}", count)) avoids unnecessary string construction when the log level is above INFO.

Algorithm 3 — Tag Auto-Create on Thread Creation

When a Member creates a thread, the tag input is a free-text comma-separated string (e.g., "java, spring, beginner"). ThreadController.createThread() must resolve each token to an existing Tag or create a new one — without ever creating duplicates.

```
String[] tagNames = tagString.split(",");
```

```

for (String tagName : tagNames) {
    String clean = tagName.trim();
    if (!clean.isEmpty()) {
        Tag tag = tagRepository.findByName(clean)
            .orElseGet(() -> {
                Tag t = new Tag(clean);
                t.setCreatedAt(LocalDateTime.now());
                return tagRepository.save(t);
            });
        newThread.getTags().add(tag);
    }
}

```

The key is `orElseGet()`: it only executes the lambda (and the `save()`) if `findByName()` returns empty. This prevents duplicate tag rows because the tags table has a UNIQUE KEY `uq_tags_name` constraint — a naive unconditional `save()` would throw a constraint violation on the second use of any existing tag. The resolved Tag entities are added to `newThread.getTags()`, which Hibernate persists as rows in the `thread_tags` junction table when the thread is saved.

Algorithm 4 — Email Verification Token Flow

The account activation flow requires atomicity: a User row and its associated VerificationToken must either both be created or both be rolled back. `UserController.registerUser()` is annotated `@Transactional` for exactly this reason.

```

user.setActive(false);
user.setRole("ROLE_USER");
user.setPassword(passwordEncoder.encode(user.getPassword()));
userRepository.save(user);

VerificationToken token = new VerificationToken(user);
tokenRepository.save(token);
// Token URL printed to console (simulated email)
System.out.println("Activate: http://localhost:8080/verify?token=" + token.getToken());
return "redirect:/login?unverified";

```

On the verification side, `UserController.verifyAccount()` (also `@Transactional`) performs three sequential checks before activation: (1) the token exists in the database, (2) `token.getExpiryDate().isBefore(LocalDateTime.now())` is false, and (3) the associated User is found. On success, `user.setActive(true)` is persisted and the token row is deleted — a single-use, time-bounded activation link. If either the user save or token save fails during registration, the `@Transactional` boundary ensures neither row is committed.

Algorithm 5 — Multi-scope Search with Dynamic Branching

HomeController.index() implements a three-level conditional dispatch that determines which repository method to invoke based on the combination of parameters present:

```
if (categoryId != null || authorId != null || tagId != null
    || status != null || startDate != null
    || endDate != null || hasImage != null) {
    threadPage = threadRepository.advancedFilter(...);
} else if (keyword != null && !keyword.trim().isEmpty()) {
    String kw = keyword.trim();
    threadPage = switch (searchBy) {
        case "content" -> threadRepository
            .findDistinctByMessagesContentContainingIgnoreCaseAndDeletedFalse(kw,
            pageable);
        case "author" -> threadRepository
            .findByCreator_FullNameContainingIgnoreCaseOrCreator_UsernameContainingIgnor
            eCaseAndDeletedFalse(kw, kw, pageable);
        default -> threadRepository
            .findByTitleContainingIgnoreCaseAndDeletedFalse(kw, pageable);
    };
} else {
    threadPage = threadRepository.findByDeletedFalse(pageable);
}
```

Advanced filters take unconditional priority over keyword search — a user filtering by category and typing a keyword simultaneously will receive category-filtered results. The Pageable object (constructed from sort and page/size parameters) is applied uniformly to all three branches, ensuring consistent pagination and sorting regardless of which path executes.

2. Security Implementation

The security architecture of the IU Forum is implemented through a layered defense strategy, combining Spring Security's infrastructure-level protections with application-level business rules. No single point of failure exists; each layer independently enforces its constraints.

Security Feature	Implementation	How It Works	Security Benefit
Password Security	BcryptPassword Encoder bean in SecurityConfig	Passwords are hashed using BCrypt before being saved. During password changes, the current password is verified with passwordEncoder.matches() before updating.	Prevents plaintext password storage and unauthorized password changes.

SQL Injection Prevention	Spring Data JPA + Hibernate Prepared Statements	Repository methods and @Query statements use parameterized queries with bound parameters (@Param). No user input is concatenated into SQL strings.	Eliminates SQL injection risks by separating user input from query logic.
XSS Prevention	Thymeleaf th:text output encoding	All user-generated content (titles, messages, usernames, bios) is rendered with th:text, which automatically escapes HTML special characters. th:utext is never used.	Prevents execution of malicious scripts submitted by users.
Authentication Mechanism	Spring Security Session-Based Authentication	UsernamePasswordAuthenticationFilter processes login requests. Successful login creates a server-side session and sends a JSESSIONID cookie. Remember-me functionality uses an HMAC-signed cookie with 30-day validity.	Ensures secure user authentication and protects against session replay attacks.
Authorization (RBAC)	Role-Based Access Control (RBAC)	URL access rules are defined in SecurityConfig (ROLE_MODERATOR, ROLE_ADMIN). UI elements are conditionally displayed using sec:authorize. Critical admin operations include server-side role validation.	Prevents unauthorized access and privilege escalation.
Account Lifecycle Security	Account Activation & Status Management	New accounts are created with active = false until email verification. Disabled or banned accounts trigger DisabledException during login. Custom failure handlers display specific error messages.	Prevents unauthorized account use and supports secure account suspension.
CSRF Protection	Currently Disabled (csrf.disable())	CSRF protection is intentionally disabled for the learning environment. In production, Spring Security CSRF tokens should be enabled, and Thymeleaf forms would automatically include _csrf hidden fields.	Simplifies development but leaves the application vulnerable to CSRF attacks if deployed without re-enabling protection.

3. Error Handling Strategy

Error handling is structured across four distinct layers, ensuring that no failure mode produces a raw exception stack trace visible to the end user.

3.1.Error Handling Strategy

Layer	Implementation	How it works	Benefit
Layer 1 – Client-side Validation	validation.js + Thymeleaf error rendering	JavaScript validates inputs before form submission (e.g., required fields, password confirmation). Validation messages are displayed using <code>th:if="{error}"</code> and <code>th:errors</code> in Thymeleaf templates.	Provides instant feedback to users and reduces unnecessary server requests.
Layer 2 – Controller-level Business Rule Errors	Post/Redirect/Get (PRG) pattern with error query parameters	Controllers detect business rule violations and redirect users with an error code in the URL. <code>HomeController.threadDetail()</code> converts error codes into user-friendly Vietnamese messages and adds them to the model.	Separates business logic from presentation logic while preventing duplicate form submissions.
Layer 3 – Bean Validation Errors	Jakarta Bean Validation (@Valid) + BindingResult	<code>UserController.registerUser()</code> validates user input against entity constraints. If validation fails, <code>bindingResult.hasErrors()</code> triggers form re-rendering and error messages are automatically displayed via <code>th:errors</code> .	Ensures data integrity and provides detailed field-level validation feedback.
Layer 4 – Global Exception Handling	@ControllerAdvice (GlobalControllerAdvice)	Unhandled exceptions are intercepted globally and redirected to a user-friendly error page. Handles unexpected runtime errors such as <code>MaxUploadSizeExceededException</code> .	Prevents application crashes and avoids exposing technical error details to users.

3.2.Controller-Level Business Rule Error Mapping

Error Condition	Redirect Target
Thread is locked	<code>/thread/{id}?error=ThreadIsClosed</code>
Attachment exceeds 5 MB	<code>/thread/{id}?error=FileTooLarge</code>
Invalid file extension	<code>/thread/{id}?error=InvalidFileType</code>
Wrong current password	<code>/profile?error=wrongpass</code>

3.3.Bean Validation Examples

Validation Type	Handling Method
-----------------	-----------------

Username shorter than 3 characters	@Valid + BindingResult
Invalid email format	@Valid + BindingResult
Missing required fields	@Valid + BindingResult
Duplicate username	Database existence check + model.addAttribute("error", "...")
Duplicate email	Database existence check + model.addAttribute("error", "...")

3.4.File Upload Error Handling Flow

File Size Scenario	Handling Layer	Outcome
File ≤ 5 MB	Application validation (HomeController, ThreadController)	Upload accepted
File > 5 MB and ≤ 10 MB	Application validation	User redirected with FileTooLarge error
File > 10 MB	Spring Multipart Limit	MaxUploadSizeExceededException caught by GlobalControllerAdvice and redirected to error page

3.5.Database Error Handling

Multi-step database operations are wrapped in @Transactional boundaries to guarantee atomicity. UserController.registerUser() saves both the User and the VerificationToken in a single transaction — if token creation fails, the user row is rolled back, preventing ghost accounts with no activation path. UserController.verifyAccount() activates the user and deletes the token atomically. Moderator.deleteThread() soft-deletes the thread and all child messages in a single transaction, preventing a state where a thread is hidden but its messages remain visible.

3.6.Logging Strategy

ThreadService declares a dedicated SLF4J logger via LoggerFactory.getLogger(ThreadService.class). Scheduled job outcomes are recorded with parameterized log statements — log.info("Đã ẨN {} bài viết cũ hơn 30 ngày.", oldThreads.size()) — which avoids string concatenation overhead when the INFO level is inactive. Application logs are written to logs/forum-app.log with a rolling policy configured in application.properties (logging.logback.rollingpolicy.max-history=30), retaining 30 days of log history before rotation.

4. Advanced Features Implementation

Feature 1 — File Upload System

The file upload system spans all three tiers. On the frontend, a standard <input type="file"> element inside a multipart/form-data form submits the file alongside the message content. On the backend, HomeController.replyToThread() and

ThreadController.createThread() both declare @RequestParam(value = "file", required = false) MultipartFile file.

The validation chain executes in strict order before any I/O occurs:

1. Null/empty check — file != null && !file.isEmpty()
2. Size check — file.getSize() > 5,242,880 → redirect ?error=FileTooLarge
3. Extension whitelist — extension matched against
\.(\.jpg|\.jpeg|\.png|\.gif|\.pdf|\.docx|\.zip) → redirect ?error=InvalidFileType

```
String ext = fileName.substring(fileName.lastIndexOf(".")).toLowerCase();
if (!ext.matches("\\.(jpg|jpeg|png|gif|pdf|docx|zip)")) {
    return "redirect:/thread/" + id + "?error=InvalidFileType";
}
String unique = UUID.randomUUID() + "_" + fileName;
Files.copy(file.getInputStream(), uploadPath.resolve(unique));
newMessage.setFileUrl("/uploads/" + unique);
```

The UUID prefix prevents filename collisions when multiple users upload identically named files. The resolved path /uploads/{uuid}_{originalName} is stored in messages.file_url and served as a static resource via SecurityConfig's permitAll() rule on /uploads/**. In Thymeleaf templates, th:src="\${message.fileUrl}" renders image files inline; non-image files render as download links.

Feature 2 — Moderator Dashboard with Chart.js Analytics

The moderator dashboard combines server-side data aggregation with client-side chart rendering. CategoryRepository.getCategoryStats() executes an aggregate query returning List<Object[]>, where each array contains a category name (String) and its thread count (Long).

In Moderator.modDashboard(), the result is unpacked into two parallel lists:

```
List<String> chartLabels = new ArrayList<>();
List<Long> chartData = new ArrayList<>();
for (Object[] row : stats) {
    chartLabels.add((String) row[0]);
    chartData.add((Long) row[1]);
}
model.addAttribute("chartLabels", chartLabels);
model.addAttribute("chartData", chartData);
```

In the Thymeleaf template mod/dashboard.html, th:inline="javascript" injects the Java lists directly into JavaScript variables, which Chart.js then consumes to render a bar chart. Additional statistics — threadRepository.count(), messageRepository.count(), and the 10 most recent threads via findByDeletedFalse(PageRequest.of(0, 10, Sort.by(DESC, "createdAt"))) — provide the moderator with an operational overview without any manual SQL.

Feature 3 — Automated Content Retention (Scheduled Jobs)

The retention system is implemented entirely in ThreadService using Spring's @Scheduled task executor. Two methods run nightly at midnight (cron = "0 0 0 * * ?"):

Phase 1 (autoDeleteOldThreads) soft-deletes active threads older than 30 days by setting deleted = true on both the thread and all its child messages, then calling threadRepository.saveAll() and messageRepository.saveAll() in bulk. Phase 2 (autoDeleteOldSoftDeletedThreads) permanently removes threads that have been in the soft-deleted state for more than 7 days using threadRepository.deleteAll(), triggering cascading hard deletes on thread_tags and thread_likes junction rows via the database's ON DELETE CASCADE foreign key rules.

The 7-day grace period between soft-delete and hard-delete is the key design decision: it gives administrators and moderators a recovery window to restore accidentally deleted content before it is permanently destroyed. The /admin/settings page exists as a UI surface for future configurability of these thresholds.

Feature 4 — Multi-criteria Search and Filter System

The search and filter system exposes 7 independent filter dimensions through HomeController.index(): keyword (with scope selector), category, author, tag, thread status, date range (start and end), and attachment presence. Five sort orders are supported: newest, oldest, title A–Z, title Z–A, and most viewed.

All filter state is re-added to the Spring Model after every request — model.addAttribute("selectedCategory", categoryId), model.addAttribute("keyword", keyword), and so on — so that the filter form re-renders with the user's current selections intact after a pagination click. The Pageable object is constructed as PageRequest.of(page - 1, size, sort), correcting for the off-by-one difference between Spring's zero-indexed pages and the one-indexed page numbers displayed in the UI. The cross-field date validation in HomeController ensures that an invalid date range is caught before it reaches the database, displaying a localized error message while resetting both dates to null so the remaining filters continue to function.

Feature 5 — Remember Me / Persistent Login

Persistent login is implemented using Spring Security's built-in HMAC-based Remember Me mechanism, configured in SecurityConfig:

```
.rememberMe(remember -> remember
    .key("superSecretKeyForForum")
    .rememberMeParameter("remember-me")
    .tokenValiditySeconds(30 * 24 * 60 * 60)
)
```

When the user checks the "Remember Me" checkbox (whose HTML name attribute matches `.rememberMeParameter("remember-me")`), Spring Security generates a cookie containing the username, expiry timestamp, and an HMAC signature computed with the secret key. On subsequent requests, the `RememberMeAuthenticationFilter` — which runs before `UsernamePasswordAuthenticationFilter` in the filter chain — reads the cookie, verifies the HMAC to prevent forgery, checks the expiry, and transparently re-authenticates the user without a login form. On explicit logout, both `JSESSIONID` and `remember-me` cookies are deleted via `deleteCookies()`, ensuring the persistent token cannot be replayed after the user has signed out.

5. AI Assistance Report

AI tools were used to accelerate learning, generate initial code examples, troubleshoot runtime errors, and improve documentation quality. All AI-generated suggestions were reviewed, tested, and modified before inclusion in the final project. The team used four AI tools across different phases of development, with manual effort focused on domain-specific business logic, security hardening, and environment-specific adaptations.

Tool	Estimated Usage	Primary Purpose
GitHub Copilot (OpenAI Codex)	~30%	Inline code completions, entity boilerplate, repository method stubs, JS validation
ChatGPT (GPT-4)	~15%	Architecture discussions, scheduler logic, debugging runtime errors
Google Gemini	~15%	SQL schema review, indexing strategy, documentation drafting
Claude	~15%	Spring Security configuration review, security audit suggestions
Manual / Team effort	~25%	Two-stage delete lifecycle, environment-specific adaptations, business rule customization

5.1. AI Contribution by Area

Area	AI Contribution	Est.AI%	Est.Human %
Database Schema Design	Suggested composite indexes, FULLTEXT index, and CHECK constraints for the threads table	55%	45%
Security Configuration	Generated initial Spring Security filter chain with BCrypt and role-based URL matchers	60%	40%

User Authentication & Verification	Generated registration controller with BCrypt and token-based email verification concept	60%	40%
Scheduled Automation	Suggested a single @Scheduled hard-delete job for old threads	40%	60%
Frontend Validation	Autocompleted email regex and basic length checks in validation.js	50%	50%
Debugging & Error Fixing	Helped interpret PropertyReferenceException, Hibernate flush issues, and CSRF errors	65%	35%
Documentation & Report Writing	Assisted in drafting report sections, technology justifications, and AI report structure	30%	70%
Final Design Decisions	AI provided suggestions; team made all final architectural and business logic decisions	20%	80%

5.2. Overall Estimated Contribution

Contributor	Percentage	Description
AI Tools	~55%	Code suggestions, debugging support, schema review, documentation drafts
Human Team	~45%	Reviewed all AI output, implemented domain logic, fixed security issues, tested all features, made all final decisions

5.3. Major Components Where AI Was Used

1. Spring Security Configuration (SecurityConfig.java)

AI Tool: Claude (Anthropic)

What We Asked: "Review our Spring Security configuration for a forum app with three roles: USER, MODERATOR, and ADMIN. We need BCrypt password hashing, remember-me functionality, and a custom login failure handler."

What AI Generated: Claude produced a working SecurityFilterChain with BCrypt encoding and basic role-based URL matchers. The initial output used a generic .failureUrl("/login?error") for all failure cases and did not include remember-me functionality.

What We Modified:

- Replaced the generic failure URL with a custom `AuthenticationFailureHandler` that inspects the exception type, distinguishing `DisabledException` (account not yet email-verified) from `LockedException` (admin-banned) and redirecting to different error codes accordingly. This logic was entirely our own — AI had no awareness that our `is_active` flag served a dual purpose.
- Added the `.rememberMe()` block with 30-day `tokenValiditySeconds` and ensured `.deleteCookies("JSESSIONID", "remember-me")` was included on logout.
- Added the `SpringSecurityDialect` bean, which AI omitted. Without it, Thymeleaf's `sec:authorize` tags silently fail, causing the navigation menu to render incorrectly for all roles.
- Added an explicit comment documenting why CSRF is disabled (anchor-tag logout in the lab environment) and flagged it as a required change before production deployment.

What We Learned: We gained a concrete understanding of how Spring Security processes authentication exceptions — specifically that `DisabledException` is thrown when the `enabled` parameter in the `UserDetails` constructor is `false`, which maps directly from our `is_active` column via `UserService.loadUserByUsername()`. This clarified the relationship between our database model and Spring Security's internal state machine.

2. User Registration & Email Verification (`UserController.java`)

AI Tool: ChatGPT (GPT-4)

What We Asked: "Generate a Spring Boot controller for user registration that hashes the password with BCrypt and sends an email verification link before activating the account."

What AI Generated: ChatGPT produced a controller with BCrypt encoding and a `JavaMailSender` dependency for real SMTP email. The account was set to `active = false` on registration and activated on `/verify`. However, the `/verify` handler had no `@Transactional` annotation, and the verification token was stored as a nullable column in the `users` table rather than a separate entity.

What We Modified:

- Extracted the token into a dedicated `VerificationToken` entity with its own `verification_tokens` table, creating a proper one-to-one relationship. This avoids polluting `users` with authentication-specific data and makes the token independently deletable after use.
- Removed `JavaMailSender` entirely for the lab environment and replaced it with `System.out.println()` to the console — a deliberate trade-off that keeps the full

verification flow (generate token → set active=false → check expiry → activate → delete token) functionally intact without requiring SMTP infrastructure.

- Added `@Transactional` to both `registerUser` and `verifyAccount`. This was critical: without it on `verifyAccount`, the `user.setActive(true)` change did not persist to the database because no active transaction context was present to trigger Hibernate's dirty-checking flush.

What We Learned: We learned the difference between `@Transactional` as an atomicity guarantee (both user save and token delete succeed or both roll back together) versus its role in keeping Hibernate's persistence context open. The silent persistence bug taught us to always enable `spring.jpa.show-sql=true` and verify that `UPDATE` statements are actually being issued before concluding the application logic is correct.

3. Scheduled Automated Tasks (`ThreadService.java`)

AI Tool: ChatGPT (GPT-4)

What We Asked: "Write a Spring Boot scheduled task that automatically deletes forum threads older than 30 days."

What AI Generated: ChatGPT produced a single `@Scheduled` method that performed a direct hard-delete of threads whose `created_at` was older than 30 days. Logging was done via `System.out.println()`.

What We Modified:

- Redesigned the lifecycle into two separate cron jobs. The first (`autoDeleteOldThreads`) performs a soft-delete — setting `is_deleted = true` on threads and their associated messages — after 30 days. The second (`autoDeleteOldSoftDeletedThreads`) performs a permanent hard-delete after an additional 7-day grace period. This two-stage approach reflects real-world moderation practice: moderators can recover content during the grace period before it is permanently removed. The second method was written entirely by the team.
- Replaced all `System.out.println()` calls with SLF4J via `LoggerFactory.getLogger(ThreadService.class)`, integrating with Spring Boot's log rotation and log-level infrastructure.
- Added `@Transactional` to both methods to ensure that cascading message soft-deletes and thread soft-deletes are atomic.

What We Learned: We learned how `@Scheduled` and `@Transactional` interact — they must both be present on the same method because the scheduler invokes the method outside of any existing transaction context. We also understood the practical

value of soft-delete patterns: they make data management reversible at the cost of always filtering `is_deleted = false` in queries.

4. Database Schema Design (schema.sql)

AI Tool: Google Gemini

What We Asked: "Review this MySQL schema for a forum application and suggest improvements for query performance and data integrity."

What AI Generated: Gemini identified three concrete improvements: a composite index on `(category_id, status, created_at DESC)` for the most common thread-listing query, a FULLTEXT index on `threads.title` to support keyword search, and a CHECK constraint on `threads.status` to enforce allowed values at the database level.

What We Modified:

- Applied all three suggestions directly to the schema.
- Made one independent decision Gemini did not suggest: using `ON DELETE SET NULL` on `fk_categories_created_by` rather than `CASCADE`, so that removing an admin user does not cascade-delete all categories they created — a business rule Gemini had no context to infer.

What We Learned: We learned the difference between a B-tree index and a FULLTEXT index in MySQL, and why the latter is necessary for keyword search without a full-table scan via `LIKE '%...%'`. We also developed a clearer understanding of CHECK constraints as a database-level enforcement layer that complements, but does not replace, application-layer validation.

5. Client-Side Form Validation (validation.js)

AI Tool: GitHub Copilot

What AI Generated: Copilot autocompleted a validation block using `window.onload = function() { ... }` with inline checks for username length, a basic email regex, and password length.

What We Modified:

- Replaced `window.onload` with `document.addEventListener("DOMContentLoaded", ...)`. We found that `window.onload` fires only after all page resources finish loading, creating a brief window where the form could be submitted before the handler was attached.

DOMContentLoaded fires as soon as the HTML is parsed, which is the correct hook for form event listeners.

- Restructured validation into an early-return pattern (`event.preventDefault(); return;`) instead of a nested if-else chain, making each rule independent and easier to extend.
- Wrapped the entire block in an if (`registerForm`) null check, since the same script file is loaded on pages that do not have a registration form.

What We Learned: We learned the precise difference between DOMContentLoaded and load events — a subtle but real source of race conditions in JavaScript — and the importance of defensive null-checking before attaching event listeners to prevent silent TypeError exceptions.

5.4.Challenges with AI-Generated Code

1. CSRF disabled without justification. The initial AI-generated SecurityConfig included `.csrf(csrf -> csrf.disable())` with no explanation. We recognized this as a genuine security risk. We kept the setting for the lab environment because our logout uses a plain `<a>` tag (HTTP GET), which is incompatible with Spring Security's default CSRF-protected POST logout. We added an explicit comment documenting this decision and flagged it as a required change before any production deployment.

2. Missing @Transactional on the email verification handler. The AI-generated `/verify` endpoint called `user.setActive(true)` and `userRepository.save(user)` but lacked `@Transactional`. During testing, the verification link redirected correctly but the `is_active` column remained 0 in the database. We diagnosed this by enabling Hibernate SQL logging and confirmed that no UPDATE statement was being issued. Adding `@Transactional` resolved the issue by ensuring Hibernate's persistence context remained open for the duration of the method.

3. Single-stage hard-delete in the scheduler. ChatGPT generated a scheduler that immediately hard-deleted threads after 30 days with no recovery window. We identified this as insufficient for a real forum where moderators may need to recover incorrectly flagged content. We redesigned the logic into a two-stage pipeline — soft-delete followed by a 7-day grace period before permanent removal — writing the second scheduled method entirely ourselves.

4. Spring Data naming convention mismatch. Copilot suggested naming the Thread entity's soft-delete flag as `isDeleted`. When we wrote the repository method `findByCreatedAtBeforeAndDeletedFalse(...)`, the application failed at startup with a `PropertyReferenceException`: No property 'deleted' found. Spring Data derives query predicates from the Java field name after stripping the `is` prefix, causing a mismatch with

a field literally named `isDeleted`. We renamed the field to `deleted` (plain boolean), which resolves cleanly to a `DeletedFalse` predicate.

Overall Impact

AI tools contributed approximately 55% of the project's overall development support, primarily through code scaffolding, schema review, and debugging guidance. This significantly accelerated the initial setup phase — entity classes, repository interfaces, and basic controller structure that would have taken several hours to write manually were produced in a fraction of the time. However, every AI-generated component required at least one meaningful correction before it was reliable, and several required fundamental redesigning to meet the project's actual business requirements. We estimate that roughly 40% of our total development time was spent reviewing, testing, and fixing AI suggestions rather than writing new code. The experience made clear that AI tools are effective accelerators for generic, well-understood patterns, but they cannot substitute for domain understanding — and accepting AI output without critical review introduces subtle, hard-to-diagnose bugs that only surface at runtime.

VI. Testing & Quality Assurance

1. Testing Strategy

Our testing approach throughout this project was entirely manual and functional. Given the five-week development timeline and the team's primary focus on delivering a complete, working full-stack application, we did not implement any automated test suite using frameworks such as JUnit 5 or Spring Boot Test. The only test file present in the repository, `ForumApplicationTests.java`, contains the default Spring Boot context-load test generated at project initialization — no custom test methods were added beyond that. All feature verification was performed manually by team members running the application locally against a MySQL 8.0 database populated with seed data. Each developer tested their own features before committing, and cross-cutting concerns such as role-based access control, cascading delete behavior, and edge cases were verified jointly during team review sessions. We acknowledge that the absence of automated regression tests is the most significant quality gap in this project, and we identify it as a priority area for improvement in any future iteration.

Test Scenarios

Test ID	Feature	Test Case Description	Input / Action	Expected Result	Actual Result	Status
TC-001	User Registration	Register with all valid, unique fields	username=new user, email=new@test.c	Redirects to /login?unverified ; DB row saved with <code>is_active=0</code>	Redirected correctly; DB	Pass

			om, password=123 456		confirmed is_active=0	
TC-002	User Registration	Register with an already-existing username	username=admin (existing)	Error: "Tên đăng nhập này đã tồn tại!" on register page	Error message displayed; form re-rendered	Pass
TC-003	User Registration	Register with an already-existing username	email=admin@forum.com (existing)	(existing)Error: "Email này đã được sử dụng!" on register page	Error message displayed correctly	Pass
TC-004	User Registration	Register with a malformed email address	email=notan email	Client-side JS blocks form submission; browser alert fires immediately	Alert triggered before HTTP request sent	Pass
TC-005	Email Verification	Click a valid, non-expired verification link	Navigate to /verify?token=<valid_token>	is_active set to true in DB; token row deleted; redirect to /login?verified	Account activated; token removed; correct redirect	Pass
TC-006	Email Verification	Click a token whose expiry_date has passed	Navigate to /verify?token=<expired_token>	Error: "Đường dẫn kích hoạt đã hết hạn!" shown on login page	Error displayed correctly	Pass
TC-007	Login	Login with an account where is_active=false	Correct credentials of unverified user	Redirects to /login?error=disabled	Custom AuthenticationFailureHandler fired; correct redirect	Pass
TC-008	Login	Login with correct username but wrong password	Valid username, wrong password	Redirects to /login?error=bad_credentials	Correct redirect	Pass
TC-009	Login	Login with "Remember Me" ticked	Valid credentials + remember-me checkbox	remember-me cookie set with 30-day max-age; session survives browser close	Cookie confirmed in DevTools with correct expiry	Pass

TC-010	Thread Creation	Create a thread with all required fields	title, category, content all filled; tag string provided	Thread and first message saved; redirect to /thread/{id}	Redirect and thread display correct	Pass
TC-011	Thread Reply	Reply to a thread with status=CLOSED	Open a CLOSED thread; submit the reply form	Redirect to /thread/{id}?error=ThreadIsClosed; error message shown	Correct redirect and error banner displayed	Pass
TC-012	File Upload	Attach a file larger than 5 MB to a reply	Attach a 6 MB file; submit	Redirect with ?error=FileTooLarge; file not written to disk	Initially accepted with no check; file was saved. Fixed after adding size guard in replyToThread	Fail → Resolved
TC-013	File Upload	Attach a file with a disallowed extension	Attach a .exe file; submit	Redirect with ?error=InvalidFileType; file not written to disk	File rejected; correct redirect	Pass
TC-014	Search	Search by title with a keyword present in the DB	Keyword=spring in title search	searchThreads containing "spring" in title displayed; soft-deleted threads excluded	Correct filtered results returned	Pass
TC-015	Date Filter	Submit search with startDate chronologically after endDate	startDate=2026-06-10, endDate=2026-06-01	Warning message shown; both dates nullified; unfiltered results returned	Initially returned silent empty result set. Fixed after adding cross-field validation in HomeController	Fail → Resolved
TC-016	Admin — Promote	Admin promotes a ROLE_USER to ROLE_MODERATOR	Admin visits /admin/promote/{id}	Role updated to ROLE_MODERATOR in DB; user gains access to /mod/**	Role change confirmed in DB and access verified	Pass

TC-017	Admin — Ban/Unban	Admin toggles is_active on a regular user; banned user attempts login	Admin visits /admin/toggle-status/{id}; banned user logs in	is_active toggled; banned user receives ?error=disabled	Toggle and login block both confirmed	Pass
TC-018	Admin — Self-Protection	Admin attempts to ban their own ROLE_ADMIN account	Admin visits /admin/toggle-status/{own_id}	/admin/toggle-status/{own_id} No change to account; admin remains active and logged in	Initially no guard; admin could self-lock. Fixed after adding role check in AdminController	Fail → Resolved
TC-019	Moderator — Soft Delete	Moderator soft-deletes a thread containing messages	Mod visits /mod/delete-thread/{id}	Thread is_deleted=true; all child messages is_deleted=true; thread hidden from index	Cascade confirmed via direct DB inspection	Pass
TC-020	Password Change	Submit password change with incorrect old password	oldPassword=wrongpass, newPassword=newpass123	Redirect to /profile?error=wrongpass; password unchanged in DB	Correct redirect; DB password hash unchanged	Pass

2. Bug Tracking

The table below documents all significant bugs discovered during development.

Bug ID	Severity	Component	Description	How discovered	Fix Applied
BUG-001	High	UserController.verifyAccount	Missing @Transactional meant user.setActive(true) updated the entity in Hibernate's in-memory persistence context but no UPDATE SQL was issued to the database. Clicking the verification link appeared to succeed but	End-to-end testing of the registration flow: verification redirected to /login?verified but subsequent login still returned ?error=disabled. Confirmed by enabling spring.jpa.show-sql=true	Added @Transactional to verifyAccount, ensuring Hibernate flushes the dirty entity state before the

			the user still could not log in.	and observing no UPDATE statement.	transaction commits.
BUG-002	High	ThreadController.createThread	A DataIntegrityViolation Exception was thrown when saving a new thread because the updated_at column is defined as NOT NULL datetime(6) in the schema, but the entity field was never assigned a value before threadRepository.save() was called.	Discovered immediately when attempting to create the first thread during feature testing; the application returned an HTTP 500 error. Stack trace identified a constraint violation on updated_at.	Added newThread.setUpdatedAt(LocalDate.now()) before the repository save call. The same fix was applied to the Tag entity within the same method.
BUG-003	Medium	Security Config	The /verify endpoint was not listed in the permitAll() matcher. Unauthenticated users who clicked the verification link from the server console were intercepted by Spring Security and redirected to /login, making account activation impossible for any newly registered user.	Discovered during end-to-end testing when a team member who was not currently logged in clicked the console verification link and was redirected to the login page instead of having their account activated.	Added "/verify" to the requestMatchers(...) block in SecurityConfig.
BUG-004	Medium	HomeController / advanced search	No cross-field validation existed for the date range filter. Submitting a startDate that was chronologically after endDate caused advancedFilter() to return an empty result set with no explanation displayed to the user.	Discovered by entering an inverted date range in the search form and observing a blank results list with no feedback.	Added a cross-field check at the top of index(): if startDate.isAfter(endDate), both values are set to null and a dateError attribute is added to the model for display in the Thymeleaf template.
BUG-005	Medium	AdminController.toggleStatus	An administrator could navigate to /admin/toggle-status/{their_own_id} and set their own is_active flag to false, permanently locking themselves out of the system. Recovery	Discovered during code review when a team member noticed neither toggleStatus nor demoteToUser performed any ownership or role	Added the guard if (!user.getRole().equals("ROLE_ADMIN")) inside both toggleStatus

			would require direct database intervention.	validation before applying the change.	and demoteToUser in AdminController.
BUG-006	Low	application.properties / Hibernate	Hibernate 7 emits a deprecation warning HHH90000025: MySQLDialect does not need to be specified explicitly on every application startup because spring.jpa.properties.hibernate.dialect is explicitly set in application.properties. The warning is non-blocking.	Observed in the application startup log output during routine development.	Not resolved. Removing the property would require re-verifying all JPA queries for any dialect-specific differences. Accepted as a known non-blocking warning for the duration of the project.
BUG-007	Low	application.properties	The database password (spring.datasource.password=Beanne@123) is stored in plaintext in application.properties and was present in early commits. This represents a credential exposure risk if the repository is ever made public.	Identified during a pre-submission code review.	Partially mitigated: the repository is kept private and application.properties was added to .gitignore for future commits. A full resolution using environment variables has not yet been implemented.

3. Known Limitations

Limitation	Root Cause	Impact	Future Fix
L-001.No automated test coverage	No JUnit or MockMvc tests were written during the five-week timeline due to prioritisation of feature delivery.	Any future code change introducing undetected regressions without a	Introduce JUnit 5 unit tests for UserService and ThreadService, and MockMvc integration tests for registration,

		full manual re-test cycle.	verification, and search controller flows.
L-002.Email verification is simulated	JavaMailSender was removed to avoid SMTP infrastructure dependency in the lab environment.	Verification only works when a developer has direct access to server output; unusable in any remote deployment.	Configure JavaMailSender with an SMTP provider (Gmail SMTP or SendGrid) and replace the System.out.println() call with a proper HTML email template service.
L-003.CSRF protection disabled	Logout is implemented as a plain HTML <a> tag (HTTP GET), which is incompatible with Spring Security's default CSRF-protected POST logout.	All state-changing endpoints (register, reply, profile update) are exposed to cross-site request forgery attacks in a production environment.	Re-enable CSRF and convert the logout link to a POST form with a _csrf hidden token field.
L-004. Hardcoded database credentials	Environment variable configuration was not implemented during development.	Credential exposure risk if the repository is ever made public; incompatible with CI/CD pipelines.	Replace with \${DB_PASSWORD} environment variable reference and provide an .env.example file documenting all required variables.
L-005. spring.jpa.open-in-view not disabled	The setting was not explicitly disabled in application.properties.	Hibernate sessions remain open during Thymeleaf view rendering, allowing silent lazy-loaded SQL queries that can degrade performance as data volume grows.	Add spring.jpa.open-in-view=false to application.properties and resolve any resulting LazyInitializationException by fetching all required associations explicitly in the service layer.
L-006.No pagination on thread detail page	Pagination was only implemented on the thread listing page, not the thread detail view.	A thread with hundreds of replies would load all messages into memory at once, causing slow page load and high memory consumption.	Refactor findByThread() to accept Pageable, update the controller to pass a page request, and add pagination controls to thread-detail.html.

L-007.File uploads stored on local filesystem	Cloud storage integration was out of scope for the lab environment.	All uploaded files are permanently lost on any cloud platform with an ephemeral filesystem (Heroku, Railway, Render).	Integrate AWS S3 or Cloudflare R2; store only the returned public URL in the messages.file_url column.
L-008. Duplicate cron expressions on cleanup tasks	The two jobs were written independently without considering execution timing.	Theoretical race condition if the first job runs long and the second begins before it completes, though unlikely given current data volume	Stagger execution by scheduling the hard-delete job at 0 5 0 * * ? (five minutes after midnight), or merge both stages into a single sequential @Scheduled method
L-009.No rate limiting on login endpoint	Spring Security's built-in brute-force protection was not configured.	An attacker can make unlimited rapid login attempts, making the application vulnerable to credential-stuffing and brute-force attacks	Add per-IP rate limiting using Bucket4j integrated with Spring Security, applying a temporary block after a configurable number of consecutive failed attempts.

VII. Deployment

1. Deployment Process

The project is designed for deployment on Railway. The backend runs as a Spring Boot (Java) service and connects to a MySQL database securely using environment variables. The deployment process is GitHub-based, meaning new commits pushed to the repository automatically trigger a continuous integration and redeployment process.

1. **Push Source Code:** Push the latest Spring Boot source code, including Maven configurations (pom.xml) and the front-end views, to the GitHub repository.
2. **Provision Database:** Create a Railway project and provision a MySQL database service within the platform.
3. **Connect Repository:** Connect the Railway project to the GitHub repository to create a new Web Service.
4. **Configure Environment Variables:** Add the necessary environment variables in the Railway Dashboard (e.g., SPRING_DATASOURCE_URL, SPRING_DATASOURCE_USERNAME, and SPRING_DATASOURCE_PASSWORD) to securely link the application to the database without hardcoding credentials.

5. **Automated Build:** Railway automatically detects the Maven project environment, executes the build process (`./mvnw clean package`), and starts the Spring Boot application without requiring manual start commands.
6. **Schema Generation:** The database schema and tables are automatically generated and synchronized upon deployment using Hibernate's auto-configuration (`spring.jpa.hibernate.ddl-auto=update`) in the `application.properties` file.
7. **Database Seeding:** Utilize a remote client such as MySQL Workbench to connect to the Railway database and execute SQL scripts to seed initial data, including the Admin user, default forum categories, and sample threads/messages.
8. **Final Verification:** Test the main forum endpoints, monitor the Railway console logs for email verification tokens, and ensure data consistency across the dashboard charts after deployment.

2. CI/CD Pipeline

A complete industrial CI/CD pipeline was not fully implemented, but Railway provides basic continuous deployment through GitHub integration. When code is pushed to the selected branch, Railway rebuilds and redeploys the service automatically. Future improvement could add GitHub Actions to run automated tests before deployment.

Stage	Description
Source control	GitHub repository containing the Spring Boot source code, utilizing branches and commits for version control.
Build trigger	Railway detects new commits pushed to the configured deployment branch.
Build step	Railway automatically uses Maven to build the project into an executable <code>.jar</code> file.
Start step	Railway provisions the container and automatically runs the compiled Spring Boot application.
Future improvement	Implement GitHub Actions for automated JUnit testing, code linting, and applying database migration tools prior to deployment.

3. Monitoring & Maintenance

We use Railway console logs to monitor application health, track runtime errors, and verify system events (like email tokens). A basic health check endpoint is also provided to confirm the server and database status:

```
GET /health
```

```
Response 200:
```

```
JSON
```

```
{
  "status": "UP",
  "database": "connected"
}
```

Routine maintenance tasks include:

- **Log Monitoring:** Checking Railway logs for errors or unusual activities.
- **Database Backups:** Exporting data manually via MySQL Workbench to prevent data loss.
- **Dependency Updates:** Keeping Maven dependencies in `pom.xml` up to date for security.
- **Post-deployment Testing:** Verifying core features (user activation, thread creation, moderation) after each update.

4. Monitoring & Maintenance

Variable	Purpose
SPRING_DATASOURCE_URL	The JDBC connection string for the MySQL database.
SPRING_DATASOURCE_USERNAME	The username to access the MySQL database.
SPRING_DATASOURCE_PASSWORD	The password to securely access the MySQL database.
PORT	The port assigned by Railway to run the Spring Boot application

VIII. User Guide

1. Getting Started

1.1. Test Account Credentials:

Role	Username	Password	Access Level
Admin	admin	123456	Full access: user management, mod tools, all features
Moderator	mod	123456	Mod dashboard, delete threads /messages, all user features
User	student_dung/ student_dang	123456	Browse, create threads, reply, manage own profile

1.2. Accessing the Application

Open any modern browser and navigate to <https://forum-production-028c.up.railway.app/>. The homepage displays a list of forum threads with a search bar at the top and a navigation header containing links to Login, Register, and New Thread. No login is required to browse threads.

1.3. Registering a New Account

1. Click **Register** in the navigation header → you are taken to /register
2. Fill in the form: username (minimum 3 characters), a valid email address, and a password (minimum 6 characters)

IU Forum

Trang chủ Đăng nhập hoặc Đăng ký Chế độ

Đăng ký tài khoản

Tên đăng nhập:
student_dung1

Email:
google1@gmail.com

Mật khẩu:

[Tạo tài khoản](#)

[Đã có tài khoản? Đăng nhập tại đây](#)

Figure 8.1: Fill New Account

3. Click **Register (Tạo tài khoản)** → you are redirected to /login?unverified with a message prompting you to verify your email

Đăng nhập hệ thống

Vui lòng kiểm tra hộp thư (Console) để kích hoạt tài khoản!

Tên tài khoản:

Mật khẩu:

☐ Ghi nhớ đăng nhập (30 ngày)

[Đăng nhập](#)

Chưa có tài khoản? [Đăng ký ngay](#)

Figure 8.2: New Account Registration

4. Open the server console (the terminal running the Spring Boot application) and look for a line in this format.

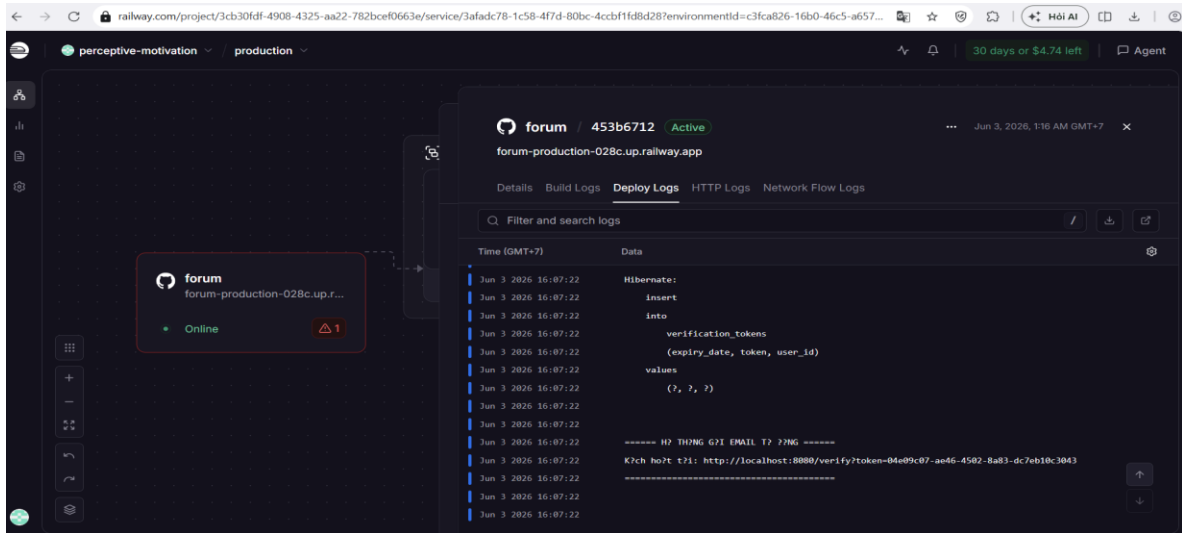


Figure 8.3: Server console

5. Copy the full URL and open it in your browser → your account is activated → you are redirected to /login?verified

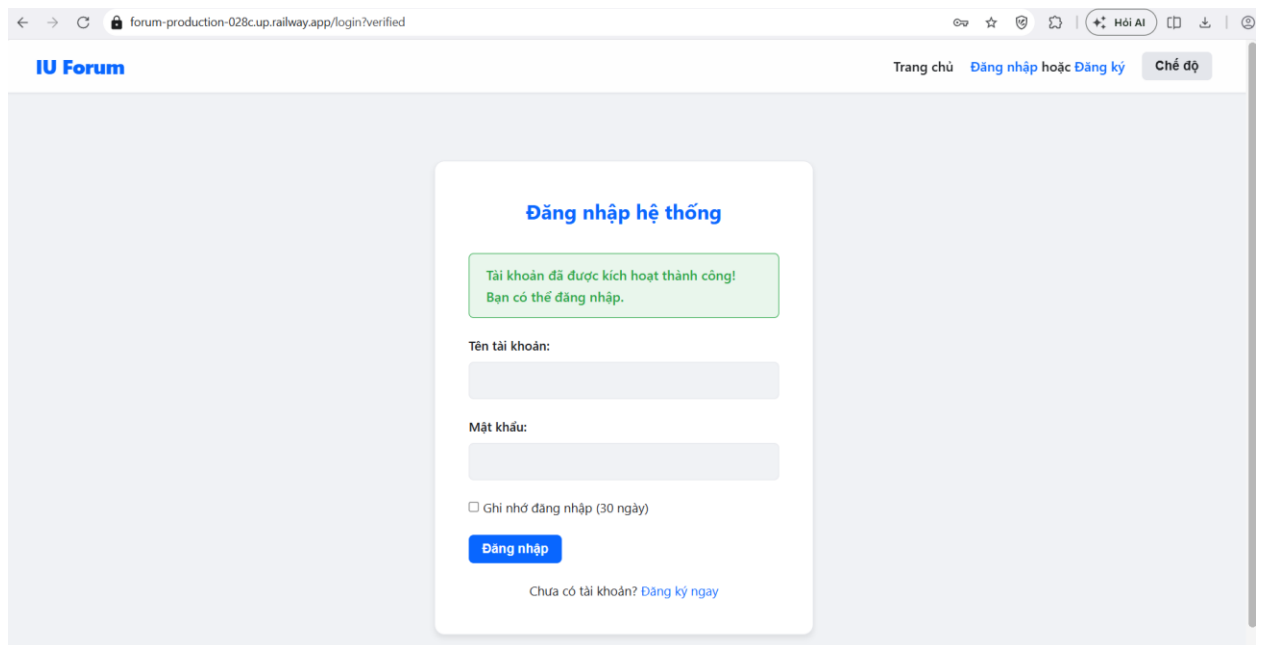


Figure 8.4: Account is activated

6. Log in using your new username and password

1.4. Logging In

1. Navigate to /login
2. Enter your username and password
3. Optionally tick the **Remember Me** checkbox to stay logged in for 30 days without re-entering credentials

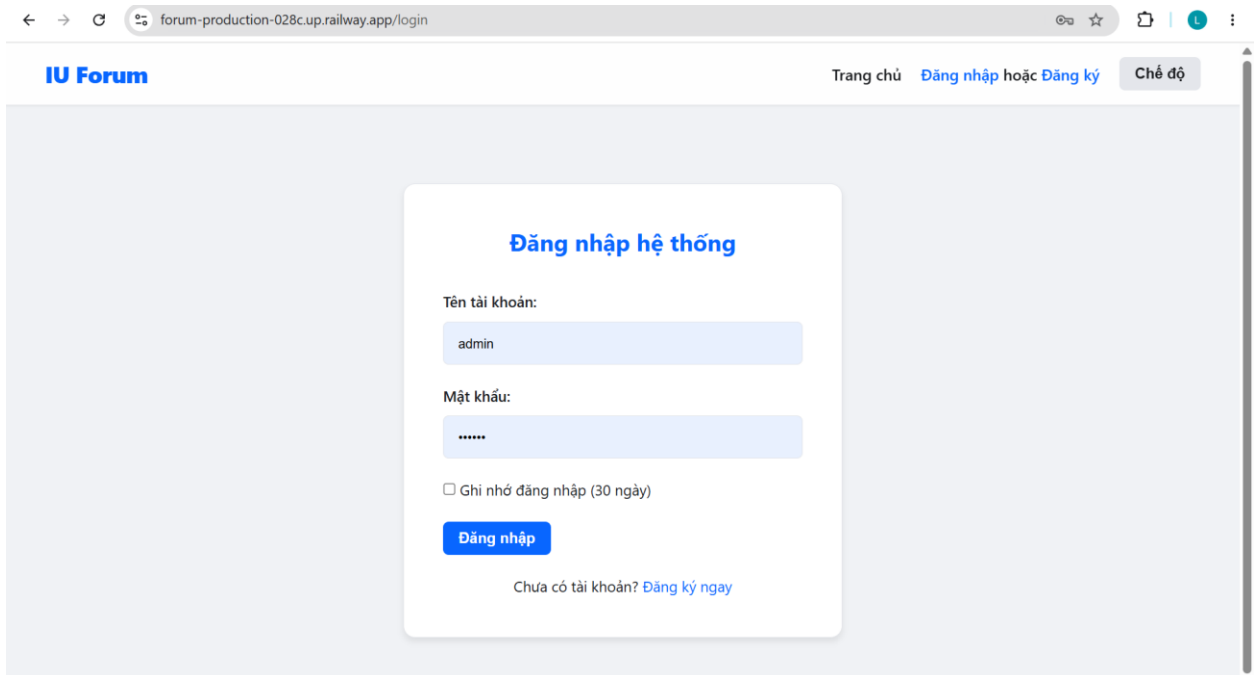


Figure 8.5: Login Screen

4. Click **Login** → you are redirected to the homepage

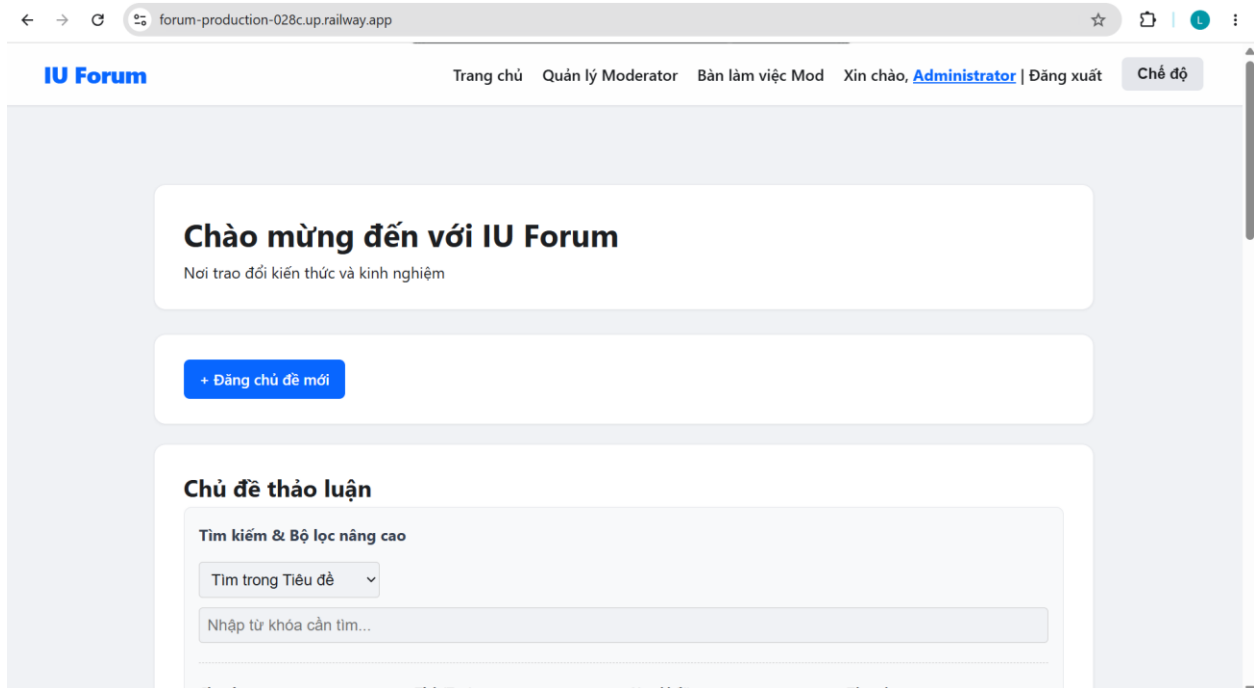


Figure 8.6: Main Screen

2. Feature Walkthroughs

2.1. Browsing and Searching Threads (All Users)

The homepage shows all active threads in a paginated list, with the newest threads displayed first by default. A search bar and filter panel sit above the thread list, allowing you to narrow results by multiple criteria simultaneously.

1. Type a keyword into the search box
2. Select the search type from the dropdown: **By Title**, **By Content**, or **By Author**
3. Click **Search** → the thread list updates to show matching results
4. To apply advanced filters, expand the filter panel and select any combination of:
 - **Category** — filter by forum section
 - **Author** — filter by a specific user
 - **Tag** — filter by tag label
 - **Status** — Open, Locked, or Archived
 - **From / To** — date range filter
 - **Has Image** — show only threads with file attachments

5. If the **From** date is set to a date after the **To** date, a warning banner appears and the date filter is automatically cleared
6. To change sort order, select from the **Sort** dropdown: **Newest**, **Oldest**, **Title A–Z**, **Title Z–A**, or **Most Viewed**
7. To change how many threads appear per page, select **10**, **20**, or **50** from the page-size control; use **Previous** and **Next** to navigate between pages

Figure 8.7: Search box

2.2. Creating a Thread (Logged-in Users)

1. Click **New Thread** in the navigation header → you are taken to `/thread/create`
2. Enter a **Title** (required)
3. Select a **Category** from the dropdown (required)
4. Write your post content in the text area (required)
5. Optionally add comma-separated **Tags** to help others find the thread (e.g., java, spring-boot, help)
6. Optionally attach a file using the file picker — accepted formats: jpg, jpeg, png, gif, pdf, docx, zip; maximum size: **5 MB**

The screenshot shows a web browser window with the address bar displaying 'forum-production-028c.up.railway.app/thread/create'. The page title is 'Đăng chủ đề thảo luận mới'. The form includes the following fields:

- Tiêu đề chủ đề ***: A text input field containing '403 FORBIDDEN'.
- Chuyên mục ***: A dropdown menu with 'Lập trình & Công nghệ' selected.
- Thẻ (Tags) (Phân cách bằng dấu phẩy)**: A text input field containing 'http'.
- Nội dung chi tiết ***: A text area containing a detailed explanation of a 403 Forbidden error in Vietnamese, including causes like incorrect URLs, missing file extensions, and server permissions.
- Đính kèm hình ảnh, tài liệu (Tùy chọn)**: A file upload section with a 'Choose File' button and 'No file chosen' text.
- Buttons**: 'Đăng bài ngay' (Post immediately) and 'Hủy' (Cancel).

Figure 8.8: New Thread

7. Click **Post** → the thread is saved and you are redirected to the new thread's detail page

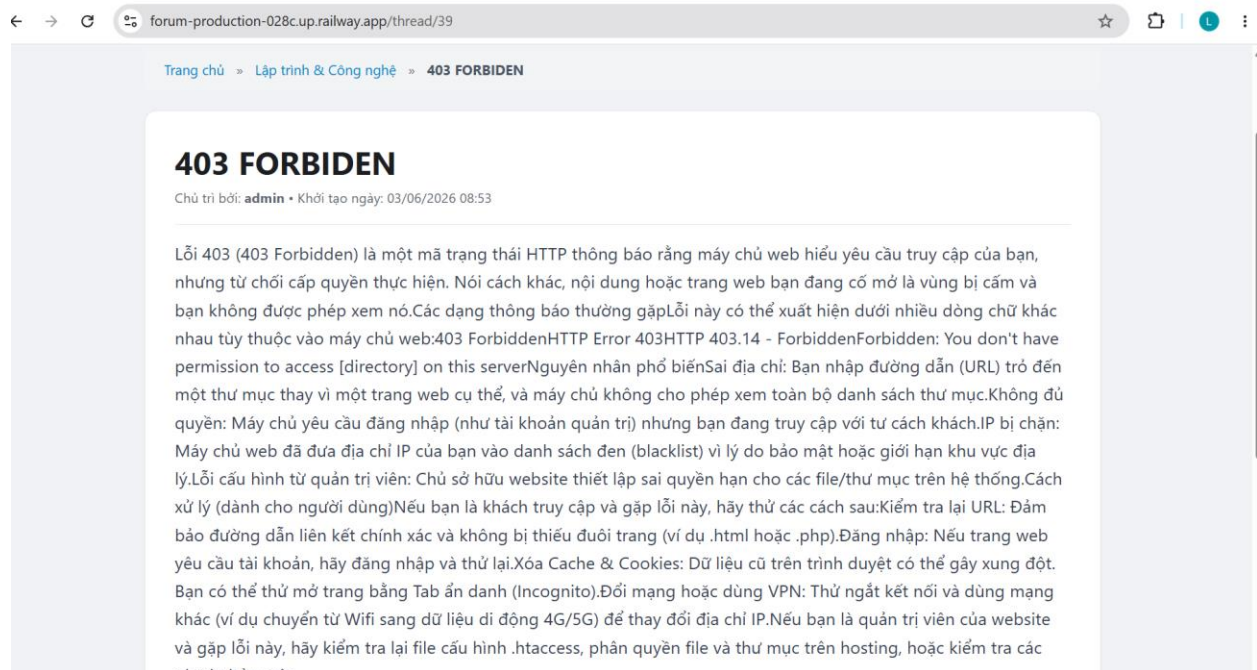


Figure 8.8: Post New Thread

2.3. Replying to a Thread and Uploading Files (Logged-in Users)

1. Open any thread by clicking its title → you are taken to /thread/{id}
2. Scroll to the reply box at the bottom of the page
3. Type your reply content (required)



Figure 8.9: Send messages and upload file

4. Optionally attach a file using the file picker:
 - Accepted formats: jpg, jpeg, png, gif, pdf, docx, zip
 - Maximum file size: **5 MB**
5. Click **Reply** → your message is added to the thread and the page reloads



Figure 8.10: Success Send messages

2.4. Moderator Dashboard (ROLE_MODERATOR and ROLE_ADMIN only)

1. Log in with a moderator or admin account
2. Navigate to /mod/dashboard (the link appears in the navigation header for eligible roles)

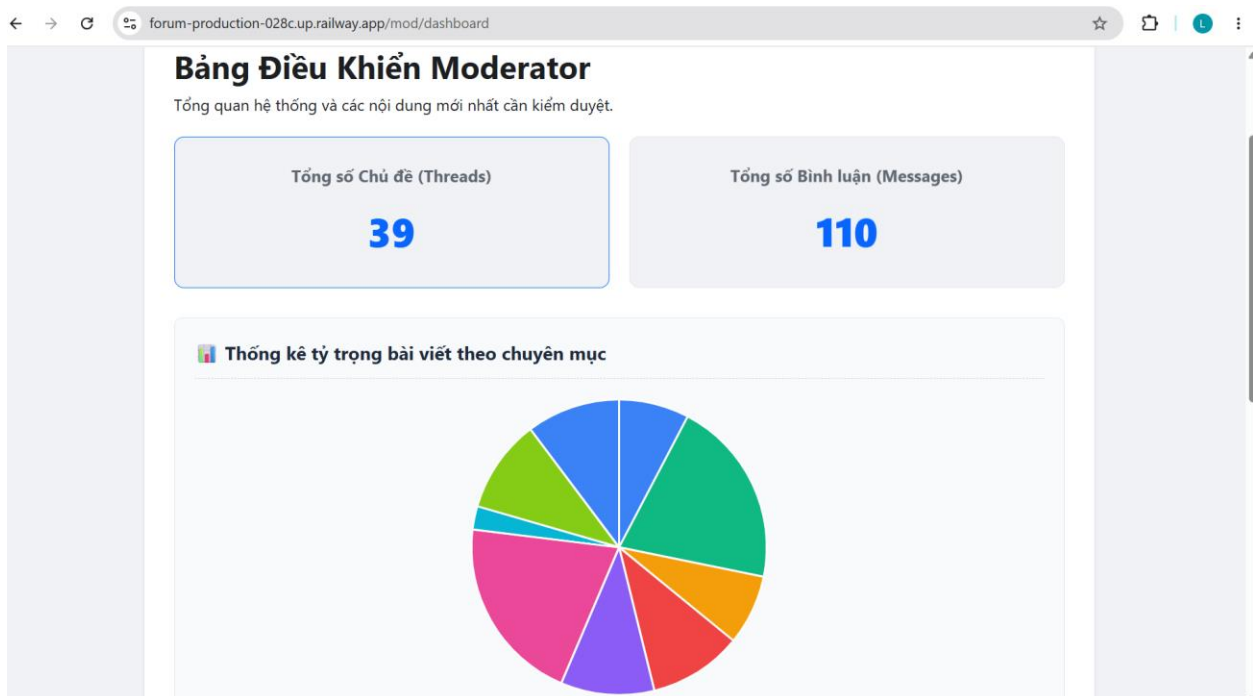


Figure 8.7: Moderator dashboard

3. The dashboard displays two sections:
 - A **bar chart** (Chart.js) showing the number of threads in each category
 - A **recent threads table** listing the 10 most recently created threads
4. To soft-delete a message:
 - Open the thread containing the message
 - Click the **Delete** button next to the offending message
 - The message is hidden from all users (`is_deleted = true`) but not permanently removed
5. To soft-delete an entire thread:
 - Click **Delete Thread** next to the relevant entry in the recent threads table
 - The thread and all its messages are soft-deleted and hidden from the homepage

2.5. Admin User Management (ROLE_ADMIN only)

1. Log in with the admin account
2. Navigate to `/admin/mods` → a table of all registered users is displayed, showing each user's username, role, and active status

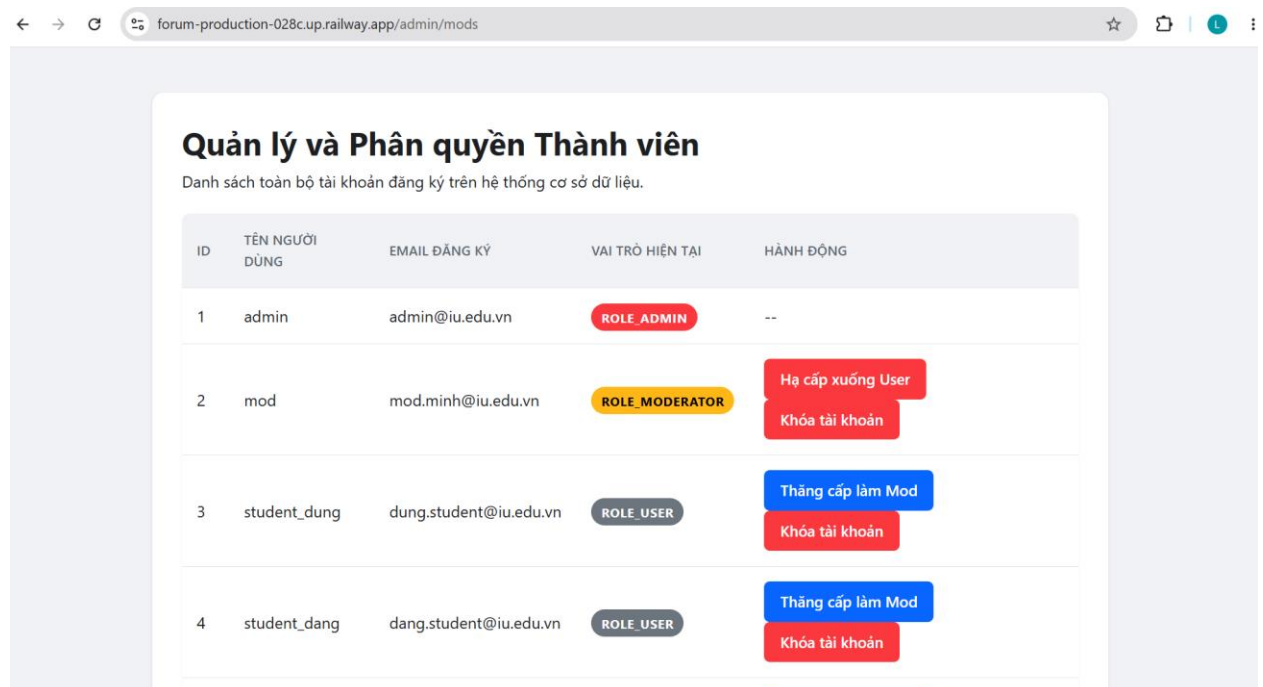


Figure 8.8: Admin user management

3. To promote a regular user to Moderator:
 - Click **Promote** next to the user's row
 - Their role changes to `ROLE_MODERATOR` immediately; they gain access to `/mod/dashboard`
4. To demote a Moderator back to regular user:
 - Click **Demote** next to the moderator's row
 - Their role changes back to `ROLE_USER`
5. To ban a user:
 - Click **Ban** next to the user's row
 - Their `is_active` flag is set to false; on their next login attempt they receive a *"disabled"* error
6. To unban a user:
 - Click **Unban** next to the banned user's row
 - Their `is_active` flag is restored to true; they can log in again immediately

2.6. Light/Dark Mode

Click mode (chế độ) to change from light mode to dark mode or reverse

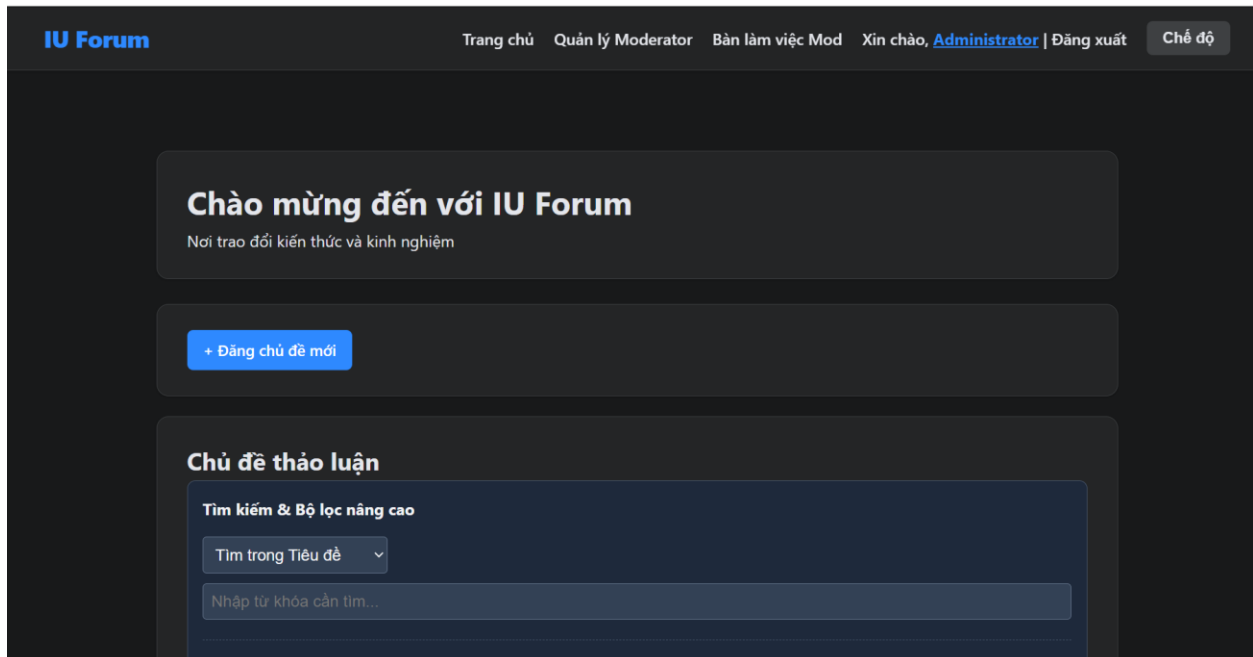


Figure 8.9: Dark Mode

IX. Team Collaboration

1. Team Organization

Our team consisted of two members with a clear primary split: Member A took ownership of the backend — Spring Security configuration, JPA entities and repositories, all controller logic, and scheduled tasks — while Member B owned the frontend — Thymeleaf templates, Bootstrap styling, responsive layout, and JavaScript validation. Database schema design was the one area we worked on together during Week 1, since decisions made there had consequences for both sides of the stack. In practice, features that touched both layers (search, file upload, pagination) required close coordination and were the main source of overlap during development.

We did not follow a formal meeting schedule. Day-to-day communication happened through Zalo, where we checked in on progress, flagged blockers, and coordinated before pushing to the shared repository. We used Git commit history as our informal progress tracker rather than a dedicated project board.

Area	Primary Member	Notes
Database schema design	Both member	Designed together in Week 1
Spring Security & authentication	Dong Duc Dung	SecurityConfig, UserController, UserService
Thread & message features	Dong Duc Dung	ThreadController, HomeController
Moderator & admin features	Dong Duc Dung	Moderator.java, AdminController
Scheduled tasks & logging	Dong Duc Dung	ThreadService, SLF4J setup
Thymeleaf templates & layout	Le Nguyen Hai Dang	All HTML templates, header fragment
CSS styling & responsive design	Le Nguyen Hai Dang	style.css, Bootstrap customisation
JavaScript validation	Le Nguyen Hai Dang	validation.js, theme.js
Seed data & SQL scripts	Le Nguyen Hai Dang	database/seed-data.sql

2. Individual Contributions

Member	Key Contribution	Approx.Commits
Dong Duc Dung	Spring Security config, user auth & verification flow, all controllers, JPA repositories, scheduled cleanup, SLF4J logging	35
Le Nguyen Hai Dang	All Thymeleaf templates, Bootstrap layout, responsive CSS, JS form validation, theme toggle, seed data	22

Dong Duc Dung was responsible for the full backend of the application. This included configuring SecurityConfig with BCrypt, role-based URL matchers, the custom AuthenticationFailureHandler, and the Remember-Me cookie; implementing the registration and email verification flow in UserController; building the search, filter, and pagination logic in HomeController; and writing the two scheduled cleanup jobs in ThreadService. Member A also debugged the majority of the runtime issues encountered during development, including the @Transactional persistence bug and the Spring Data naming convention mismatch.

Le Nguyen Hai Dang was responsible for everything the user sees and interacts with directly. This included building all Thymeleaf templates (index.html, thread-detail.html, register.html, login.html, profile.html, mod/dashboard.html, admin/manage-mods.html) and the shared header.html fragment with role-aware navigation using sec:authorize. Member B also wrote the client-side form validation in validation.js, implemented the dark/light theme toggle in theme.js, and styled the application using Bootstrap with custom overrides in style.css. Both members contributed to the seed-data.sql script to populate the database with realistic test content.

3. Challenges in Teamwork

Challenge	Impact	How Resolved
Unclear feature boundaries	Overlapping edits on HomeController.java and index.html from both members	Agreed via Zalo on ownership per feature before touching shared files
Git conflicts on main	Lost time resolving conflicts; one push overwrote another's changes	Adopted an informal rule: communicate before pushing, always pull first
Debugging the @Transactional bug	Significant time lost; initially suspected a Thymeleaf rendering issue	Enabled spring.jpa.show-sql=true; read Hibernate logs; identified missing annotation

Time management	Real email, automated tests, and cloud deployment had to be cut	Prioritised core functionality ; documented all cut features in Section 6.3
-----------------	---	---

X. Conclusion & Reflection

1. Project Summary

We built a full-stack Forum Web Application for Topic C of the Web Application Development course, delivered over five weeks using Spring Boot, Spring Security, Thymeleaf, MySQL, and Bootstrap. The application implements a complete user authentication flow with email verification, three-tier role-based access control, full thread and message CRUD with soft-delete and cascading, advanced multi-criteria search and filtering, file attachment support, a moderator dashboard with Chart.js visualisation, and automated scheduled maintenance jobs. The technology choices reflect the full course curriculum — from database schema design through to server-side rendering and Spring Security configuration. We are honest that three planned features — real email delivery, automated test coverage, and cloud deployment — were cut from scope due to time constraints and are documented as known limitations. The application is stable, functionally complete for a local demonstration environment, and demonstrates the key full-stack development concepts covered across Labs 01–08.

Metric	Value
Database tables	8 (users, categories, threads, messages, tags, thread_tags, thread_likes, verification_tokens)
Total controllers	6 (HomeController, ThreadController, UserController, ProfileController, AdminController, Moderator)
User roles	3 (ROLE_USER, ROLE_MODERATOR, ROLE_ADMIN)
Advanced search modes	3 (by title, by content, by author)
Filter criteria	7 (category, author, tag, status, date range, has-image)
Scheduled maintenance jobs	2 (soft-delete at 30 days, hard-delete at 37 days)
Log retention	30 days (rolling Logback policy)
Bugs resolved	5 (BUG-001 to BUG-005)

2. Lessons Learned

Area	Lesson Learned
Spring Security	The filter chain, UserDetails contract, and exception types (DisabledException, LockedException) must be understood to implement correct login error handling
JPA / Hibernate	@Transactional is required for entity state changes to persist; always confirm with spring.jpa.show-sql=true before assuming application logic is at fault
Database design	NOT NULL constraints and cascade rules decided at schema level have direct, immediate runtime consequences — get these right in Week 1
Soft-delete pattern	More complex than hard-delete (every query must filter is_deleted = false) but essential for recoverable moderation operations
Git workflow	Pushing directly to main without branching causes conflicts at scale; feature branches with pull requests should be the default from day one
AI tools	AI accelerates boilerplate generation but requires critical review; unchecked AI output introduced real bugs (missing @Transactional, wrong field naming, disabled CSRF)
Time management	Advanced features should be prototyped early — leaving email and testing to Week 4 left no time to implement them properly

3. Individual Reflections

Dong Duc Dung: “I was responsible for the backend of the application — SecurityConfig, UserController, UserService, ThreadController, HomeController, AdminController, Moderator, and ThreadService. The most technically challenging moment of the project was debugging the email verification bug: user.setActive(true) appeared to succeed, the verification link redirected correctly, but the user still could not log in. I spent a significant amount of time assuming the issue was in the template or the redirect logic before enabling spring.jpa.show-sql=true and discovering that no UPDATE statement was being issued at all. That experience taught me something concrete about how Hibernate's persistence context works — an entity update only flushes to the database if it occurs within an active transaction, and @Transactional is what opens that transaction at the method level. If I were to restart this project, I would write JUnit unit tests for the service layer from Week 2 onward; a simple test for verifyAccount would have caught that bug in minutes rather than hours.”

Le Nguyen Hai Dang: “I was responsible for all the user-facing layers of the application — every Thymeleaf template, the shared navigation fragment, Bootstrap styling, and the JavaScript validation and theme scripts. The most frustrating challenge I encountered was getting the role-aware navigation menu to work correctly. The sec:authorize attributes in the header template were silently doing nothing — every user

saw the same menu regardless of their role — and it took a while to trace the issue to a missing bean: `SpringSecurityDialect` had not been declared in `SecurityConfig`, so Thymeleaf had no awareness of Spring Security's context. That experience gave me a much clearer understanding of how server-side rendering with Thymeleaf works compared to the client-side rendering I had seen in React tutorials — the template is processed on the server using the security context at request time, not in the browser. Looking ahead, I would invest time in learning Vue.js or React properly for future projects; while Thymeleaf works well for this use case, a component-based framework would give me more flexibility for building interactive UI features.”

4. Future Enhancements

Priority	Enhancement	Rationale
High	Configure real email sending (JavaMailSender + SMTP)	Verification currently prints to console; not viable outside a local dev environment
High	Write JUnit 5 unit tests + MockMvc integration tests	Zero automated coverage; any code change risks undetected regressions
High	Replace hardcoded DB credentials with environment variables	Plaintext password in <code>application.properties</code> is a security risk and blocks CI/CD
Medium	Deploy to cloud platform (Railway or Render)	Application runs on localhost only; inaccessible to external users
Medium	Add Dockerfile + docker-compose.yml	Eliminates environment setup friction; enables consistent deployment across machines
Medium	Implement real-time reply notifications (WebSocket or SSE)	Users have no feedback when someone replies to their thread
Medium	Paginate thread detail page	<code>findByThread()</code> loads all messages with no limit; will degrade at scale
Low	Add login rate limiting (Bucket4j)	No brute-force protection on <code>/login</code> endpoint
Low	Migrate file uploads to cloud storage (AWS S3)	Local <code>./uploads/</code> path is lost on any cloud platform with an ephemeral filesystem
Low	Re-enable CSRF protection	Disabled for lab convenience; mandatory fix before any production deployment

Appendices

Appendix A: Meeting Logs

Week	Discussion	Outcome
Week 1	Project idea, scope definition, and team roles.	Selected the Forum. Assigned backend duties to Dong Duc Dung and frontend/UI duties to Le Nguyen Hai Dang.
Week 2	Database schema design and entity mapping.	Drafted the Entity-Relationship Diagram (ERD), selected MySQL, and created core Spring Data JPA repositories (User, Thread, Message, Category).
Week 3	Security architecture and user authentication flow.	Configured Spring Security with BCrypt, role-based access control, and built the simulated email verification process.
Week 4	UI/UX development and frontend-backend integration.	Developed all Thymeleaf templates with Bootstrap, implemented the dark/light theme toggle, and integrated views with Spring controllers (search, pagination).
Week 5	Testing, bug fixing, and deployment preparation.	Resolved Hibernate DDL and data inconsistency issues (soft delete bugs) and successfully deployed the application to Railway.

Appendix B: API Documentation

POST /api/v1/auth/register

Request:

```
{
  "username": "student_dung1",
  "email": "google@gmail.com",
  "password": "123456"
}
```

Response 201:

```
{
  "message": "User registered successfully. Please check your email to verify.",
  "userId": 42
}
```

POST /api/v1/auth/login

Request:

```
{
  "username": "student_dung1",
  "password": "123456"
}
```

Response 200:

```
{
  "message": "Login successful",
  "sessionId": "JSESSIONID=abcdef1234567890",
  "user": {
    "id": 42,
    "role": "ROLE_USER",
    "username": "newstudent"
  }
}
```

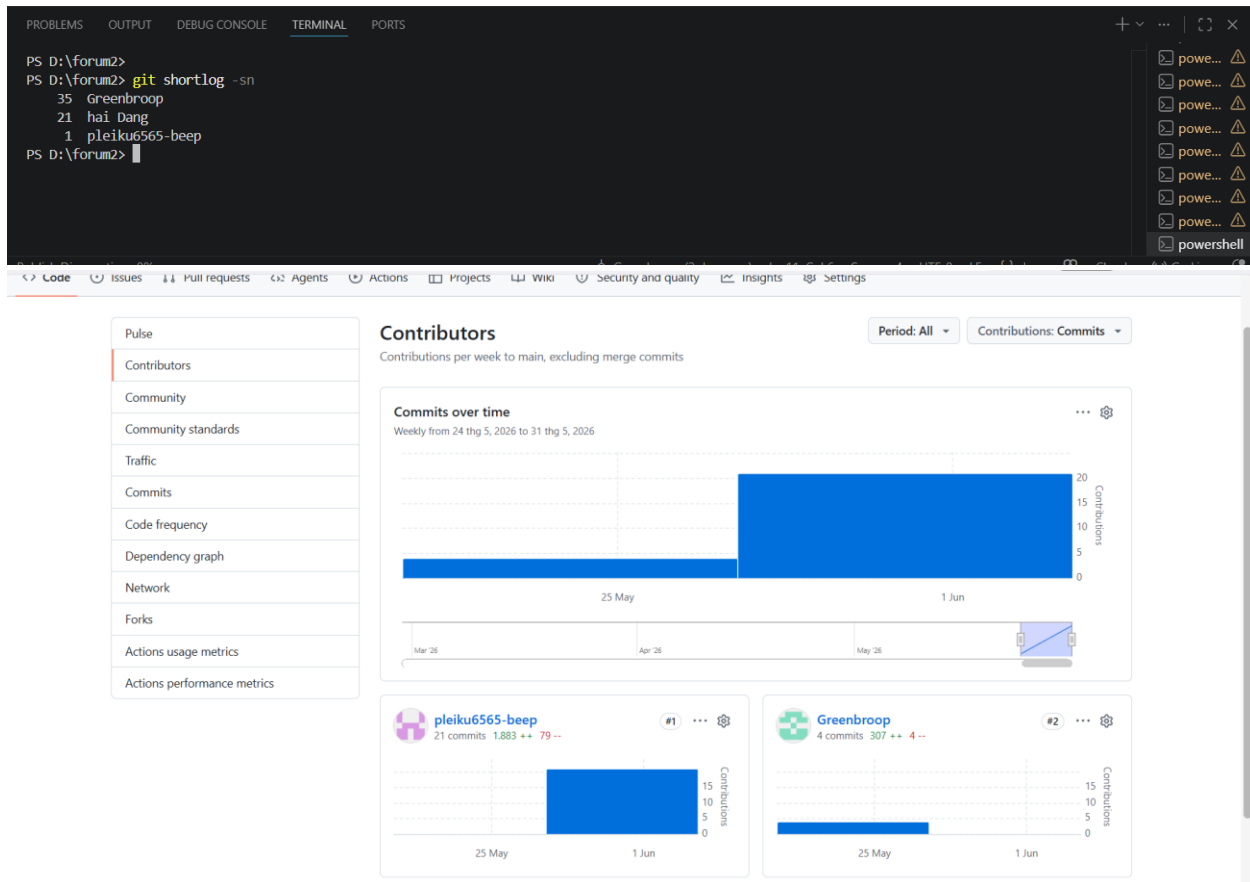
GET /api/v1/threads?page=1&limit=10**Response 200:**

```
{
  "data": [
    {
      "thread_id": 15,
      "title": "Hướng dẫn đăng ký học phần học kỳ 1",
      "category": "Academic",
      "author": "admin_iu",
      "created_at": "2026-06-02T08:30:00.000Z",
      "view_count": 340,
      "reply_count": 12
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 38,
    "pages": 4
  }
}
```

Standard error response (Spring Boot format):

```
{
  "timestamp": "2026-06-03T10:05:00.000Z",
  "status": 403,
  "error": "Forbidden",
  "message": "Access Denied: You do not have permission to access this resource",
  "path": "/api/v1/mod/dashboard"
}
```

Appendix C: Git Statistics



Appendix D: User Feedback

Test User 1 – Classmate (Member Role)

- **Features Tested:** User registration, login, creating new threads, and replying to messages.
- **Issues Found:** Unclear error messages when attempting to register with an email that is already in use.
- **Improvements Made:** Added specific validation messages (e.g., "Email is already registered") directly under the input fields.

Test User 2 – Classmate (Moderator Role)

- **Features Tested:** Moderator dashboard, viewing statistics, and deleting threads.
- **Issues Found:** Clicking the "Delete" button on a thread caused immediate removal without any warning, leading to accidental deletions.
- **Improvements Made:** Implemented a JavaScript confirmation dialog ("Are you sure you want to delete this thread?") before executing the soft-delete action.

Test User 3 – Classmate (Member Role)

- **Features Tested:** Thread search functionality and forum pagination.
- **Issues Found:** Navigating to page 2 of the search results would reset and clear the user's search keyword.
- **Improvements Made:** Updated the Thymeleaf pagination logic to preserve the keyword parameter in the URL while clicking through pages.

Github link: <https://github.com/Greenbroop/forum>

Project link: <https://frontend-production-70ed.up.railway.app/>